

Python for Finance and Optimisation

Mohamed Mrad

M2-IRFA 2026-2027

Table des matières

General Introduction	5
1 Foundations of Quantitative Finance & Data Engineering	7
Chapter Introduction	7
1.1 Microstructural Concepts : Nature of Market Data	8
1.1.1 Bid, Ask, Mid-Price and Bid-Ask Spread	8
1.1.2 Execution Price and VWAP	8
1.2 Mathematical Formalization of Returns	8
1.2.1 Non-Additivity of Arithmetic Returns vs. Additivity of Logarithmic Returns	8
1.2.2 The Asset Aggregation Trap	9
1.3 Data Engineering : Ingestion, Synchronization and Cleaning of Noisy Data	9
1.3.1 Geographical Desynchronization and the Epps Effect	9
1.3.2 Anomaly Cleaning : Rolling Z-Score Filter	9
1.3.3 Python Toolkit for Data Ingestion	10
1.4 Modeling Time-Varying Volatility : EWMA	14
1.4.1 Python Toolkit for EWMA	14
1.5 Matrix Algebra and Co-Movement Spaces	17
1.6 Case Study : Temporal Sampling Bias (Graded Assignment)	17
1.6.1 Validation Algorithm	18
1.6.2 Reflection Questions for Students	18
2 Risk Modeling, Non-Gaussian Estimation	19
Chapter Introduction	19
2.1 The Limitations of Gaussian Finance	20
2.1.1 The Stylized Facts of Mandelbrot and Fama	20
2.1.2 Mathematical Formalization of Higher-Order Moments	20
2.2 Statistical Analysis of the Distribution in Python	20
2.2.1 Python Toolbox : The <code>scipy.stats</code> Submodule	20
2.3 The Jarque-Bera normality test :	25
2.4 The Shapiro–Wilk Normality Test	26
2.5 Regulatory Risk Measures : VaR and CVaR	31
2.5.1 Mathematical Interpretation of Cornish-Fisher CVaR	35
3 Modern Portfolio Theory & Optimization	37
3.0.1 Introduction to the Markowitz Framework	37
3.0.2 Formulation 1 : Risk Minimization under a Target Return Constraint . . .	38
3.0.3 Formulation 2 : Sharpe Ratio Maximization	39
3.0.4 Formulation 3 : The Sharpe Ratio along the Efficient Frontier	41
3.0.5 Graphical Summary	42
3.0.6 SLSQP Method — Sequential Least Squares Programming	43

3.0.7	Some Remarks	44
3.0.8	Complete Scripted Implementation	45
3.1	Machine-Based Practical Work (Exercises)	52
3.1.1	Solution	52

General Introduction

Context and the Quantitative Revolution

Since the pioneering work of Harry Markowitz in 1952 on portfolio selection, the financial industry has undergone an unprecedented technological and mathematical transformation. Modern finance is no longer merely a discipline based on accounting valuation or macroeconomic intuition; it has become quantitative, algorithmic, and data-driven.

Today, asset managers, investment banks, and *hedge funds* operate in an ecosystem where information spreads at the speed of light. Faced with the proliferation of asset classes (equities, bonds, derivatives, commodities, crypto-assets), the ability to model risk, capture market co-movements, and allocate capital optimally has become a critical competitive advantage.

Why Python for Quantitative Finance ?

For several decades, the financial computing landscape was divided :

- **C++** dominated the *Front Office* for high-performance computing (exotic option pricing, high-frequency trading) due to its execution speed.
- **MATLAB** and **R** were favored by researchers (*Quants*) for statistical prototyping and econometrics.

Python has established itself as the industry's de facto standard by enabling a unique convergence. Although it is an interpreted language (theoretically slower than C++), Python relies on low-level libraries written in C/C++ (**NumPy**, **SciPy**, **Pandas**). This allows financial engineers to benefit from an expressive, readable, and extremely fast-to-develop syntax, while maintaining matrix computation performance close to native code.

Furthermore, Python's native integration with the *Machine Learning* ecosystem (**Scikit-Learn**, **PyTorch**) and its ability to handle large-scale data make it an essential tool for modern financial engineering.

Course Philosophy and Learning Objectives

The objective of this 18-hour course is to bridge the gap between the rigor of financial mathematical models and the reality of their computational implementation. Theoretical optimization often suffers from an “ivory tower” syndrome : textbook models assume perfect markets, Gaussian returns, and infinite liquidity.

This course is built around a pragmatic philosophy : **confront theory with the frictions of the real world**. The curriculum is structured around three fundamental pillars :

1. **Data Engineering and Market Infrastructure (Part I)** : Learn how to collect, clean, and synchronize heterogeneous financial data. We will examine how to handle market anomalies (*bad ticks*), deal with asynchronous market closures across global exchanges, and model the time-varying nature of volatility (EWMA).
2. **Advanced Risk Modeling (Part II)** : Move beyond the restrictive assumption of the Normal distribution. Real financial markets are characterized by frequent crashes and asymmetric behavior. We will learn how to model these “fat tails” and implement regulatory-grade risk measures (Basel III / Solvency II), such as *Value-at-Risk* (VaR) and *Expected Shortfall* (CVaR).
3. **Constrained Optimization Algorithms (Part III)** : Translate asset allocation strategies into numerical optimization problems. Beyond the classical Markowitz model, we

will code portfolios that are robust to market noise, including the Black-Litterman model, Equal Risk Contribution (*ERC*) portfolios, and approaches based on unsupervised *Machine Learning*.

At the end of this course, students will be able to design, code from end to end, and *backtest* (test against historical data) an automated and robust investment strategy suitable for a fund's production environment.

Prerequisites

To get the most out of this material, students are expected to have :

- A solid understanding of the fundamentals of Python programming (data structures, loops, functions).
- Basic knowledge of matrix algebra (vector products, matrix inversion) and probability/statistics (expectation, variance, normal distribution).

Chapitre 1

Foundations of Quantitative Finance & Data Engineering

Chapter Introduction

Professional Context and Challenges

Data infrastructure is the nervous system of quantitative finance. Before applying complex optimization models or Machine Learning algorithms, the Quantitative Developer or Risk Analyst must face a fundamental reality : market data are asynchronous, incomplete, noisy, and subject to the microstructural rules of the order book.

Poor data acquisition management or a misunderstanding of the nature of returns systematically invalidates the results of downstream models, according to the fundamental computing principle : “*Garbage in, Garbage out*” (corrupted input data lead to abnormal output results).

This chapter establishes the technical foundations of the course. We will cover the mathematical modeling of financial time, time zone management, and the essential Data Engineering tools required to build a clean, robust, and production-ready data ingestion pipeline.

Learning Objectives

By the end of this chapter, students will be able to :

1. Master the mathematical and economic distinction between arithmetic and logarithmic returns.
2. Identify and correct geographical desynchronization biases in global markets.
3. Design a complete Python pipeline to filter price anomalies (*bad ticks*) and align heterogeneous time series.
4. Model the unstable dynamics of short-term volatility using a recursive EWMA-type estimator.

Keywords

Mid-Price, Epps Effect, Log>Returns, Forward Fill, EWMA, Frobenius Norm, Rolling Z-Score.

1.1 Microstructural Concepts : Nature of Market Data

Before executing optimization algorithms, the quantitative developer must understand the infrastructure of data originating from the order book. A single price does not exist in professional markets.

1.1.1 Bid, Ask, Mid-Price and Bid-Ask Spread

At any time t , the market presents two price streams :

- P_t^{bid} (Bid Price) : The highest price at which a buyer is willing to purchase the asset.
- P_t^{ask} (Ask Price) : The lowest price at which a seller is willing to sell the asset.

By definition, $P_t^{\text{ask}} > P_t^{\text{bid}}$. The difference between these two prices is called the **Bid-Ask Spread** :

$$S_t = P_t^{\text{ask}} - P_t^{\text{bid}} \quad (1.1)$$

The spread is a direct indicator of market illiquidity. For modeling purposes, the **Mid-Price** is frequently used :

$$P_t^{\text{mid}} = \frac{P_t^{\text{ask}} + P_t^{\text{bid}}}{2} \quad (1.2)$$

1.1.2 Execution Price and VWAP

Daily databases (such as Yahoo Finance) provide the transaction closing price (*Close*). For large order volumes, portfolio managers use the **VWAP** (*Volume Weighted Average Price*), which weights each execution price by the traded volume over the period :

$$P_{\text{VWAP}} = \frac{\sum_{i=1}^M P_i \times V_i}{\sum_{i=1}^M V_i} \quad (1.3)$$

1.2 Mathematical Formalization of Returns

The evolution of an asset price P_t is modeled by a non-stationary stochastic process. Moving to returns makes it possible to work with stationary series (statistical properties that remain invariant over time).

1.2.1 Non-Additivity of Arithmetic Returns vs. Additivity of Logarithmic Returns

The arithmetic return over the period $[0, T]$, composed of T sub-periods, is :

$$R_{0 \rightarrow T} = \frac{P_T - P_0}{P_0} = \prod_{t=1}^T (1 + R_t) - 1 \quad (1.4)$$

The logarithmic return (or log-return) is defined as :

$$r_t = \ln \left(\frac{P_t}{P_{t-1}} \right) = \ln(P_t) - \ln(P_{t-1}) \quad (1.5)$$

Time additivity can be demonstrated by telescoping the sum :

$$r_{0 \rightarrow T} = \ln \left(\frac{P_T}{P_0} \right) = \ln \left(\frac{P_1}{P_0} \times \frac{P_2}{P_1} \times \cdots \times \frac{P_T}{P_{T-1}} \right) = \sum_{t=1}^T r_t \quad (1.6)$$

1.2.2 The Asset Aggregation Trap

The arithmetic return of a portfolio (R_p) is strictly the linear combination of the arithmetic returns of its N components, weighted by the weights w_i :

$$R_p = \sum_{i=1}^N w_i R_i \quad (1.7)$$

Conversely, due to the non-linearity of the logarithmic function ($\ln(A+B) \neq \ln(A) + \ln(B)$), the logarithmic return of a portfolio is **not** equal to the weighted sum of log-returns :

$$r_p \neq \sum_{i=1}^N w_i r_i \quad (1.8)$$

1.3 Data Engineering : Ingestion, Synchronization and Cleaning of Noisy Data

1.3.1 Geographical Desynchronization and the Epps Effect

When optimizing assets traded across different time zones (e.g., Paris and New York), trading windows overlap only partially. If a raw daily correlation matrix is calculated without adjustment, the coefficients are subject to an artificial downward bias known as the **Epps Effect**.

1.3.2 Anomaly Cleaning : Rolling Z-Score Filter

Real-world data feeds contain *bad ticks* (data-entry errors or abnormal price jumps caused by API bugs (Software interface bugs)). We implement a filter based on a rolling Z-Score to remove returns that deviate by more than 3 standard deviations from the local mean.

Z-Score

The **Z-Score** (or *standard score*) is a fundamental statistical tool. It measures **how many standard deviations an observation lies away from the mean of a group**.

It is an essential standardization method for comparing variables that come from completely different distributions or scales.

Practical Use and Concrete Example Imagine that you take two different exams :

- In **Mathematics**, you obtain **15/20** (the class average is 13).
- In **French**, you obtain **13/20** (the class average is 10).

To determine in which subject you actually performed better relative to the rest of the students, simply comparing the deviation from the mean is not sufficient. The dispersion of the grades (the standard deviation) must also be taken into account. This is precisely what the Z-Score allows us to do.

Mathematical Formula To calculate the Z-Score of an observation, the following formula is used :

$$Z = \frac{x - \mu}{\sigma} \quad (1.9)$$

Where :

- x is the value of the observation being studied (e.g., your grade).
- μ represents the **mean** of the population or group.
- σ represents the **standard deviation**, which measures the dispersion of the data around the mean.

Interpretation of Results The result is a real number (positive, negative, or zero) and can be interpreted as follows :

- $Z = 0$: The value is exactly equal to the group mean.
- $Z > 0$ (e.g., +1.5) : The value is above the mean. A score of +1.5 means that the observation lies 1.5 standard deviations above the mean.
- $Z < 0$ (e.g., -2) : The value is below the mean.

The Empirical Rule (Normal Distribution) For a normal distribution (Gaussian curve) :

- Approximately **68%** of observations have a Z-Score between -1 and $+1$.
- Approximately **95%** of observations have a Z-Score between -2 and $+2$.
- A Z-Score greater than $+3$ or lower than -3 is extremely rare (less than 1% probability), making it very useful for detecting anomalies or outliers (*outliers*).

1.3.3 Python Toolkit for Data Ingestion

- `yf.download()` : Queries and downloads financial data.

Example

The following line of code makes it possible to download historical data for the different financial assets under study :

```
raw_prices = yf.download(tickers, start="2022-01-01", end="2026-01-01")["Adj Close"]
```

The function `yf.download()` belongs to the `yfinance` library. It retrieves historical market data available from Yahoo Finance.

The `tickers` variable contains the different assets being studied :

```
tickers = ['AAPL', '^FCHI', '^N225', 'BTC-USD']
```

These codes correspond respectively to Apple, the CAC 40, the Nikkei 225, and Bitcoin expressed in U.S. dollars.

The argument

```
start="2022-01-01"
```

indicates the start date of the period under study, while

```
end="2026-01-01"
```

indicates the end date. Since the date specified in `end` is generally exclusive, the period considered is therefore :

$$2022-01-01 \leq t < 2026-01-01$$

The function `yf.download()` retrieves several pieces of information for each asset, including :

- `Open` : opening price ;
- `High` : highest price of the trading session ;
- `Low` : lowest price of the trading session ;
- `Close` : closing price ;
- `Adj Close` : adjusted closing price ;
- `Volume` : traded volume.

The expression

["Adj Close"]

then selects only the adjusted closing prices. The other information is therefore removed from the `DataFrame`.

The resulting structure is a dataset in which rows correspond to dates and columns correspond to the different assets :

Date	AAPL	CAC 40	Nikkei 225	BTC-USD
t_1	$P_{1,AAPL}$	$P_{1,CAC}$	$P_{1,Nikkei}$	$P_{1,BTC}$
t_2	$P_{2,AAPL}$	$P_{2,CAC}$	$P_{2,Nikkei}$	$P_{2,BTC}$
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$

The variable `raw_prices` therefore contains a `pandas DataFrame` grouping the adjusted closing prices of the different assets over the period under study.

The use of adjusted prices is particularly appropriate for return analysis, as they account for certain events affecting price series, notably dividends and stock splits (*splits*).

These data can then be used to calculate logarithmic returns :

$$r_t = \ln \left(\frac{P_t}{P_{t-1}} \right)$$

where P_t denotes the adjusted price at date t .

- `df.rolling().mean()` and `df.rolling().std()` : Compute the local statistics required for the rolling Z-Score.
- `np.abs()` : Returns the absolute value to identify two-sided deviations.
- **Filling missing values with `ffill()` and `bfill()`**

The following expression, used with the `pandas` library,

`raw_prices.ffill().bfill()`

allows missing values (NaN, meaning “Not a Number”) in a data series to be filled. It combines two successive operations :

1. `ffill()` (*forward fill*) ;
2. `bfill()` (*backward fill*).

1. `ffill()` : Forward filling. The `ffill()` method replaces each missing value with

the last known value located before it.

For example, consider the following series :

[100, NaN, NaN, 105, NaN, 110].

After applying `ffill()`, we obtain :

[100, 100, 100, 105, 105, 110].

Thus, for a position i containing a missing value, `ffill()` uses the last available observation before i .

2. `bfill()` : Backward filling.

The `bfill()` method replaces a missing value with the first known value located after it. For example :

[NaN, NaN, 100, 105, NaN]

becomes, after applying `ffill()` :

[NaN, NaN, 100, 105, 105].

The first two values remain missing because no observation is available before them. Applying `bfill()` then makes it possible to complete them :

[100, 100, 100, 105, 105].

3. Combining the two methods

The expression

`raw_prices.ffill().bfill()`

therefore performs the following operations :

`raw_prices` $\xrightarrow{\text{ffill()}}$ forward filling $\xrightarrow{\text{bfill()}}$ filling of remaining missing values.

Its behavior can be summarized as follows :

$$\text{NaN} \longrightarrow \begin{cases} \text{last known value before,} & \text{if it exists,} \\ \text{first known value after,} & \text{otherwise.} \end{cases}$$

This method is particularly useful for time series, for example when some price observations are missing. It therefore makes it possible to obtain a series containing no missing values, provided that at least one observation is known in the series.

```

1 import yfinance as yf # Yahoo Finance
2 import pandas as pd # manipulate, clean, analyze and organize data
3 import numpy as np # mainly used for numerical computation
4
5 # Ingestion of an international and multi-asset portfolio
6 tickers = ['AAPL', '^FCHI', '^N225', 'BTC-USD']
7 raw_prices = yf.download(tickers, start="2022-01-01", end="2026-01-01")['Adj
    Close']
8
9 # AAPL (Apple stock), ^FCHI (CAC 40 index),
10 # ^N225 (Japanese Nikkei 225 index),

```

```

11 # BTC-USD (Bitcoin in US dollars)
12
13 # Filling liquidity gaps (specific public holidays)
14 prices_cleaned = raw_prices.ffill().bfill()
15
16 # Calculation of log returns
17 log_rets = np.log(prices_cleaned / prices_cleaned.shift(1)).dropna()
18
19 # BAD TICK FILTER: 20-day rolling Z-Score
20 rolling_mean = log_rets.rolling(window=20).mean()
21 rolling_std = log_rets.rolling(window=20).std()
22 z_scores = np.abs((log_rets - rolling_mean) / rolling_std)
23
24 # Replace extreme returns (Z-Score > 3) with 0 (price stagnation)
25 log_rets_filtered = log_rets.mask(z_scores > 3, 0)
26
27 print(f"Number of anomalies filtered and corrected: {(z_scores > 3).sum().sum()}")

```

Listing 1.1 – Market Data Ingestion and Anomaly Filtering Pipeline

Note : Counting Outliers with sum()

The following Python expression is used to count the total number of values whose Z-Score is greater than 3 :

`(z_scores > 3).sum().sum()`

The presence of two calls to `sum()` is explained by the fact that `z_scores` is generally a **pandas DataFrame**, i.e., a two-dimensional data structure composed of rows and columns.

The condition

$$z_scores > 3$$

compares each value in the DataFrame with 3. It produces a matrix of Boolean values :

$$\begin{pmatrix} 2.1 & 4.5 & 1.2 \\ 3.8 & 1.5 & 5.2 \end{pmatrix} \longrightarrow \begin{pmatrix} \text{False} & \text{True} & \text{False} \\ \text{True} & \text{False} & \text{True} \end{pmatrix}.$$

In Python, Boolean values are interpreted as numbers when performing a sum :

$$\text{True} = 1, \quad \text{False} = 0.$$

The previous matrix can therefore be represented as :

$$\begin{pmatrix} 0 & 1 & 0 \\ 1 & 0 & 1 \end{pmatrix}.$$

First sum()

The first call to `sum()` performs a column-wise sum :

$$\begin{pmatrix} 0 & 1 & 0 \\ 1 & 0 & 1 \end{pmatrix} \xrightarrow{\text{sum}()} \begin{pmatrix} 1 & 1 & 1 \end{pmatrix}.$$

This gives the number of values greater than 3 in each column.

Second `sum()`

The second call to `sum()` then adds the results obtained for each column :

$$1 + 1 + 1 = 3.$$

We therefore obtain the total number of values greater than 3 in the entire DataFrame.
Thus,

$$(z_scores > 3).sum().sum() = \text{total number of values such that } z > 3$$

The first `sum()` performs the sum across columns, while the second adds the results obtained for all columns.

Note. If `z_scores` is a `Series` rather than a `DataFrame`, a single call to `sum()` is generally sufficient :

$$(z_scores > 3).sum()$$

Note. `dropna()` is a pandas function used to remove rows or columns containing missing values (NaN).

1.4 Modeling Time-Varying Volatility : EWMA

Return volatility is not constant ; it exhibits clusters of high and low instability (the *Volatility Clustering* phenomenon). To capture this dynamic, we formalize the **EWMA** model (*Exponentially Weighted Moving Average*), which is notably used by *RiskMetrics*.

The variance at time t , denoted by σ_t^2 , is an exponentially weighted average of the past variance and the most recent squared return :

$$\sigma_t^2 = \lambda \sigma_{t-1}^2 + (1 - \lambda) r_{t-1}^2 \quad (1.10)$$

where λ is the persistence parameter (generally set to 0.94 for daily data).

1.4.1 Python Toolkit for EWMA

- `df.ewm(alpha, adjust=False)` : Directly implements the EWMA recurrence. The parameter `alpha` is related to λ through the relationship $\alpha = 1 - \lambda$. The argument `adjust=False` ensures the strict application of the recursive formula without normalization of the initial weights.

```

1 import matplotlib.pyplot as plt
2
3 # RiskMetrics lambda parameter
4 lambda_param = 0.94
5 alpha_param = 1 - lambda_param
6
7 # Daily EWMA variance calculation
8 ewma_variance = log_rets_filtered['AAPL'].pow(2).ewm(
9     alpha=alpha_param,
10    adjust=False
11 ).mean()
12
13 # Annualization of EWMA volatility

```

```

14 ewma_vol_annualized = np.sqrt(ewma_variance * 252)
15
16 plt.figure(figsize=(12, 5))
17 plt.plot(
18     ewma_vol_annualized,
19     color='darkblue',
20     label='EWMA Volatility (RiskMetrics $\lambda=0.94$)',
21 )
22 plt.title("Time-Varying Volatility Dynamics of Apple")
23 plt.xlabel("Date")
24 plt.ylabel("Annualized Volatility")
25 plt.legend()
26 plt.grid(True)
27 plt.show()

```

Listing 1.2 – EWMA Volatility Calculation and Visualization

Explanations : Variance Estimation Using the EWMA Method

The following line of code estimates the variance of the logarithmic returns of Apple stock (AAPL) using the EWMA (*Exponentially Weighted Moving Average*) method :

```

ewma_variance = log_returns_filtered['AAPL'].pow(2).ewm(
    alpha=alpha_param,
    adjust=False
).mean()

```

This expression can be broken down into several steps.

Selecting AAPL Returns

The instruction `log_returns_filtered['AAPL']` selects the time series of logarithmic returns of Apple stock.

The logarithmic return at date t is defined as :

$$r_t = \ln \left(\frac{P_t}{P_{t-1}} \right),$$

where P_t represents the asset price at date t .

Squaring the Returns

The instruction `.pow(2)` squares each return :

$$r_t \longrightarrow r_t^2.$$

For example :

$$[-0.02, 0.01, -0.03] \longrightarrow [0.0004, 0.0001, 0.0009].$$

Squaring returns makes it possible, in particular, to measure their dispersion around zero and constitutes the basis for variance estimation.

Exponentially Weighted Moving Average

The instruction

```
.ewm(alpha=alpha_param, adjust=False)
```

sets up an exponentially weighted moving average (*Exponentially Weighted Moving Average*, EWMA).

This method assigns greater weight to recent observations than to older observations.

The EWMA variance can be represented by the recursive relationship :

$$\hat{\sigma}_t^2 = \alpha r_{t-1}^2 + (1 - \alpha) \hat{\sigma}_{t-1}^2$$

where :

- r_{t-1} represents the logarithmic return in the previous period ;
- $\hat{\sigma}_{t-1}^2$ represents the estimated variance in the previous period ;
- α is the smoothing parameter ;
- $0 < \alpha < 1$.

When α is high, recent observations have a greater influence on the variance estimate. Conversely, when α is low, older observations retain greater influence.

Calculation of the Mean

The instruction `.mean()` calculates the exponentially weighted mean of the squared returns. The EWMA variance can therefore be written as :

$$\hat{\sigma}_t^2 = \alpha r_{t-1}^2 + \alpha(1 - \alpha)r_{t-2}^2 + \alpha(1 - \alpha)^2 r_{t-3}^2 + \dots$$

The most recent observations therefore have the greatest weights.

Thus, the entire instruction

```
log_rets_filtered['AAPL'].pow(2).ewm(
    alpha=alpha_param,
    adjust=False
).mean()
```

can be summarized as :

logarithmic returns $\rightarrow$ squared returns $\rightarrow$ exponentially weighted average $\rightarrow$ EWMA variance

The variable `ewma_variance` therefore contains a time series representing the estimated conditional variance of AAPL returns.

Finally, to obtain EWMA volatility, it is sufficient to take the square root of the variance :

```
ewma_volatility = np.sqrt(ewma_variance)
```

which gives :

$$\hat{\sigma}_t = \sqrt{\hat{\sigma}_t^2}$$

EWMA volatility therefore provides a measure of volatility that assigns greater importance to recent market movements.

1.5 Matrix Algebra and Co-Movement Spaces

Let $\mathbf{r}_t = [r_{1,t}, r_{2,t}, \dots, r_{N,t}]^T$ be the vector of returns at time t . The annualized variance-covariance matrix Σ (of dimension $N \times N$) is expressed in matrix form as :

$$\Sigma = 252 \times \frac{1}{T-1} \sum_{t=1}^T (\mathbf{r}_t - \mu)(\mathbf{r}_t - \mu)^T \quad (1.11)$$

where μ is the vector of the average returns of each asset.

The Pearson correlation matrix is then given by :

$$\mathbf{R} = \mathbf{D}^{-\frac{1}{2}} \Sigma \mathbf{D}^{-\frac{1}{2}}$$

where Σ denotes the covariance matrix and $\mathbf{D}$ the diagonal matrix of variances :

$$\mathbf{D} = \text{diag}(\sigma_1^2, \sigma_2^2, \dots, \sigma_n^2).$$

Thus, each element of the correlation matrix $\mathbf{R}$ is given by :

$$\rho_{ij} = \frac{\sigma_{ij}}{\sigma_i \sigma_j}$$

where $\sigma_{ij} = \text{Cov}(r_i, r_j)$.

```

1 import seaborn as sns # statistical data visualization
2
3 # Calculation of the annualized covariance matrix
4 Sigma = log_rets_filtered.cov() * 252
5
6 # Calculation of the Pearson correlation matrix
7 R = log_rets_filtered.corr()
8
9 # Professional visualization using Seaborn
10 plt.figure(figsize=(8, 6))
11 sns.heatmap(
12     R,
13     annot=True,
14     cmap='bwr',
15     vmin=-1,
16     vmax=1,
17     linewidths=0.5
18 )
19 plt.title("Co-Movement Matrix (Correlation $R$)")
20 plt.show()

```

Listing 1.3 – Matrix Calculation of Covariance and Correlation

1.6 Case Study : Temporal Sampling Bias (Graded Assignment)

Problem : An asset management model rebalances its portfolios every month. Is it preferable to estimate the risk parameters (Σ) using daily or weekly returns ?

1.6.1 Validation Algorithm

The student must run the following script to measure the Frobenius norm of the difference matrix. The Frobenius norm measures the overall distance between two matrices :

$$\|\Sigma_{\text{Daily}} - \Sigma_{\text{Weekly}}\|_F = \sqrt{\sum_{i=1}^N \sum_{j=1}^N (\sigma_{ij,D} - \sigma_{ij,W})^2} \quad (1.12)$$

```

1 # 1. Daily Estimation
2 Sigma_daily = log_rets_filtered.cov() * 252
3
4 # 2. Weekly Estimation using specific resampling
5 prices_weekly = prices_cleaned.resample('W').last()
6 log_rets_weekly = np.log(
7     prices_weekly / prices_weekly.shift(1)
8 ).dropna()
9
10 Sigma_weekly = log_rets_weekly.cov() * 52
11
12 # 3. Calculation of the Frobenius matrix distance
13 matrix_diff = Sigma_daily - Sigma_weekly
14 frobenius_norm = np.linalg.norm(matrix_diff, ord='fro')
15
16 print("--- Quantitative Audit Results ---")
17 print("Difference matrix :\n", matrix_diff)
18 print(f"\nGlobal Frobenius distance : {frobenius_norm:.6f}")

```

Listing 1.4 – Case Study Resolution Script

1.6.2 Reflection Questions for Students

1. Why is the Frobenius distance not strictly equal to 0 even though both methods measure risk over the same historical period ?
2. What is the impact of the Epps Effect on the weekly estimation compared with the daily estimation ?

Chapitre 2

Risk Modeling, Non-Gaussian Estimation

Chapter Introduction

Professional Context and Challenges

On October 19, 1987, during « Black Monday », the Dow Jones index collapsed by 22.6% in a single trading session. Under a classical normal distribution framework, such a movement represents an event more than 20 standard deviations (20σ) away from the mean, with a probability of occurrence so small that it should not happen even on the timescale of the age of the universe. Yet the crashes of 1998 (LTCM), 2008 (Lehman Brothers), 2020 (Covid-19), or liquidity crises in crypto-assets remind us of an unavoidable mathematical reality : **real financial markets are not Gaussian**.

For a portfolio manager or banking regulator, blindly using the normal distribution is a deadly danger. It systematically leads to an underestimation of extreme loss risks. The objective of this chapter is to learn how to measure risk where it actually hides : in asymmetries and fat tails. We will formalize and implement the key indicators of institutional risk management, required by the international Basel III and Solvency II frameworks.

Learning Objectives

At the end of this chapter, students will be able to :

1. Mathematically diagnose and reject the normality assumption for an asset using the Jarque-Bera statistical test.
2. Model and interpret higher-order statistical moments (Skewness and Kurtosis) for highly volatile assets.
3. Implement in Python the algorithms for calculating *Value-at-Risk* (VaR) and *Expected Shortfall* (CVaR) using historical and parametric approaches.
4. Adjust risk calculations using the Cornish-Fisher expansion to incorporate non-Gaussian effects into capital requirements.

Keywords

Leptokurtic Distribution, Higher-Order Moments, Loss Quantile, Value-at-Risk, Expected Shortfall, Cornish-Fisher, Jarque-Bera Test.

2.1 The Limitations of Gaussian Finance

Classical financial theory (Bachelier, Markowitz, Black-Scholes) relies on the fundamental assumption that the logarithmic returns of assets follow a normal distribution (Gaussian distribution). Although this assumption greatly simplifies analytical calculations, it is systematically contradicted by real market data.

2.1.1 The Stylized Facts of Mandelbrot and Fama

As early as the 1960s, Benoît Mandelbrot and Eugene Fama highlighted the **stylized facts** of financial time series :

- **Fat Tails** : Extreme events (crashes, euphoric rallies) have a much higher probability of occurrence than predicted by the normal distribution.
- **Asymmetry (Negative Skewness)** : The return distributions of risky assets (such as stocks) often exhibit left skewness, indicating sharper downward movements than upward movements.
- **Volatility Clustering** : Volatility is not constant and propagates in clusters.

2.1.2 Mathematical Formalization of Higher-Order Moments

To quantify the deviation from normality, we analyze the centered moments of order 3 (**Skewness**) and order 4 (**Kurtosis**). Let r denote the random variable representing returns, $\mu = E[r]$ the expected value, and $\sigma = \sqrt{E[(r - \mu)^2]}$ the standard deviation. Fisher's skewness coefficient is defined as :

$$S = E \left[\left(\frac{r - \mu}{\sigma} \right)^3 \right] \quad (2.1)$$

A perfect Gaussian distribution has $S = 0$.

The kurtosis coefficient is defined as :

$$K = E \left[\left(\frac{r - \mu}{\sigma} \right)^4 \right] \quad (2.2)$$

A Gaussian distribution has $K = 3$. In quantitative finance, we use the **Excess Kurtosis** ($K_{\text{ex}} = K - 3$). If $K_{\text{ex}} > 0$, the distribution is said to be **leptokurtic** : it has fat tails and a higher central peak.

2.2 Statistical Analysis of the Distribution in Python

2.2.1 Python Toolbox : The `scipy.stats` Submodule

To audit the statistical shape of our time series, we use specialized functions :

- `stats.skew(array)` : Computes the empirical skewness coefficient. A negative value indicates a left-stretched tail (more extreme latent losses).

Explanation : In quantitative finance, this statistic is particularly useful for studying the asymmetry of return distributions.

Intuitively, skewness determines whether observations are distributed symmetrically around their mean or whether one of the two tails of the distribution is more significant than the other.

The important term in the formula is the cube $(r - \mu)^3$. Unlike the square used to calculate variance, the cube preserves the sign of the deviation from the mean.

Thus, an observation located above the mean contributes positively :

$$r - \mu > 0 \implies (r - \mu)^3 > 0$$

Whereas an observation located below the mean contributes negatively :

$$r - \mu < 0 \implies (r - \mu)^3 < 0$$

Interpretation of the Sign The sign of the skewness coefficient identifies the direction of the distribution's asymmetry.

- A positive skewness corresponds to right skewness :
- A zero skewness corresponds to an approximately symmetric distribution :
- A negative skewness corresponds to left skewness :

A negative skewness therefore means that the distribution has a **longer or heavier left tail** than its right tail.

Example Consider the following return series :

$$\{-2\%, -1\%, 0\%, 1\%, 2\%, -10\%\} \quad (2.3)$$

Most returns are relatively close to zero, but one particularly large loss of -10% is observed. This observation contributes strongly and negatively to the skewness calculation because it appears in the term :

$$(-10\%)^3 < 0$$

The cube amplifies the influence of observations that are very far from the mean. Consequently, a few strongly negative returns can lead to a significantly negative skewness.

Why Is Skewness Important in Finance? Two assets can have similar means and volatilities while exhibiting very different return distributions.

For example, one asset may exhibit relatively symmetric gains and losses, while another may exhibit many small gains but also a few extremely large losses.

In the second case, the distribution may exhibit negative skewness : $\text{Skewness}(r) < 0$. This indicates that the distribution is asymmetric toward extreme negative returns.

Skewness therefore provides information complementary to volatility. Volatility mainly measures the dispersion of returns around their mean, while skewness provides information about the **asymmetry of the distribution**.

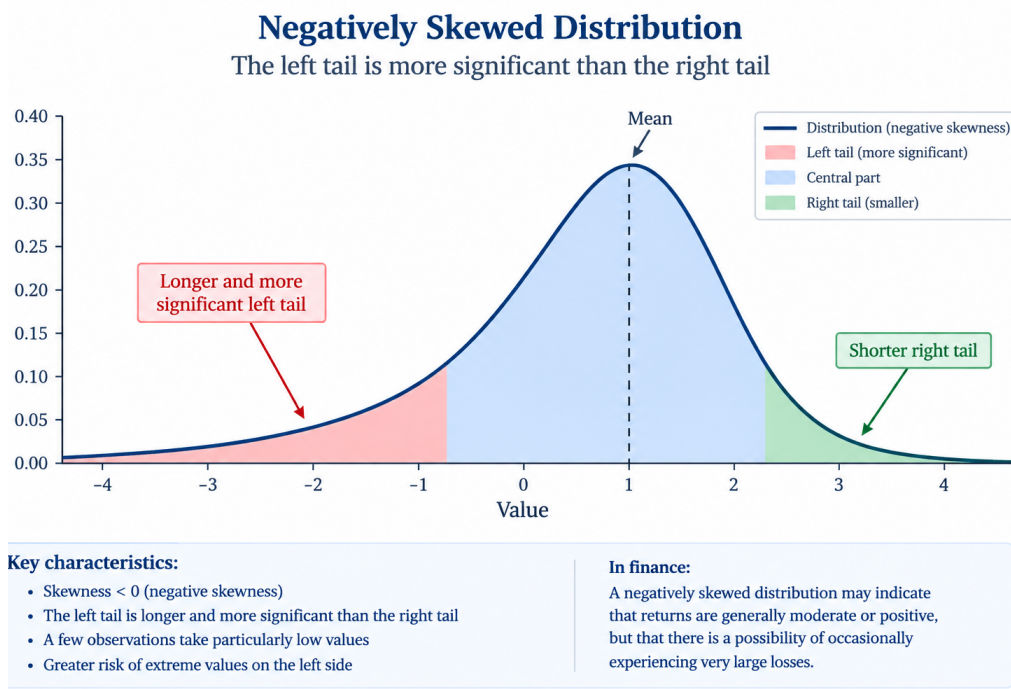


FIGURE 2.1 – Illustration of a negatively skewed distribution.

- `stats.kurtosis(array, fisher=True)` : allows the calculation of the kurtosis coefficient of a data series. In quantitative finance, kurtosis is particularly useful for studying the presence of extreme values in a return distribution. It can notably identify distributions exhibiting *fat tails*.

Example : The important term in the formula is :

$$(r - \mu)^4$$

The fourth power gives particularly strong importance to observations that are very far from the mean.

For example, consider two deviations from the mean :

$$r_1 - \mu = 1 \quad \text{and} \quad r_2 - \mu = 4.$$

With a power of two, the contributions would be :

$$1^2 = 1 \quad \text{and} \quad 4^2 = 16.$$

With a power of four, they become :

$$1^4 = 1 \quad \text{and} \quad 4^4 = 256 \tag{2.4}$$

The observation located four standard deviations from the mean therefore has a much greater influence on kurtosis. Kurtosis is thus particularly sensitive to extreme observations.

The Fisher Convention There are two conventions commonly used to define kurtosis. Pearson's kurtosis is given by :

$$K = \frac{\mathbb{E}[(r - \mu)^4]}{\sigma^4} \quad (2.5)$$

For a normal distribution, this quantity is $K = 3$. The Fisher convention consists of subtracting 3 from Pearson's kurtosis :

$$K_F = K_{ex} = \frac{\mathbb{E}[(r - \mu)^4]}{\sigma^4} - 3 \quad (2.6)$$

Thus, for a normal distribution :

$$K_F = 3 - 3 = 0 \quad (2.7)$$

This is the convention used when writing : `stats.kurtosis(array, fisher=True)`. The parameter `fisher=True` therefore instructs the function to return the **excess kurtosis** rather than Pearson's kurtosis.

Interpretation of the Result When `fisher=True`, the reference value is therefore :

$$K_F = 0$$

The result can then be interpreted as follows :

$$K_F > 0 \implies \text{fatter tails than those of a normal distribution}$$

$$K_F = 0 \implies \text{kurtosis comparable to that of a normal distribution}$$

$$K_F < 0 \implies \text{thinner tails than those of a normal distribution}$$

A distribution with positive Fisher kurtosis is generally described as **leptokurtic**.

What Does “Fat Tails” Mean ? A distribution has fat tails when it contains more observations very far from its mean than would be predicted by a normal distribution with the same mean and variance.

Consider, for example, a return series :

$$\{-10\%, -1\%, 0\%, 1\%, 12\%\}$$

The returns of -10% and 12% are very far from the mean.

Because of the fourth power present in the kurtosis formula, these observations have a significant influence :

$$(-10\%)^4 \quad \text{and} \quad (12\%)^4.$$

Thus, the presence of a few extremely large in absolute value returns can strongly increase kurtosis.

Financial Interpretation This property is particularly important when analyzing financial returns. In many financial markets, returns do not perfectly follow a normal distribution. They may exhibit **fat tails**, meaning that extreme events occur more frequently than predicted by a normal distribution.

For example, a model based on a normal distribution may assign a very low probability to a return of -8% whereas such an event may be less rare in actual financial data.

Positive kurtosis therefore highlights this characteristic :

$$K_F > 0 \implies \text{fat tails} \implies \text{more extreme observations}$$

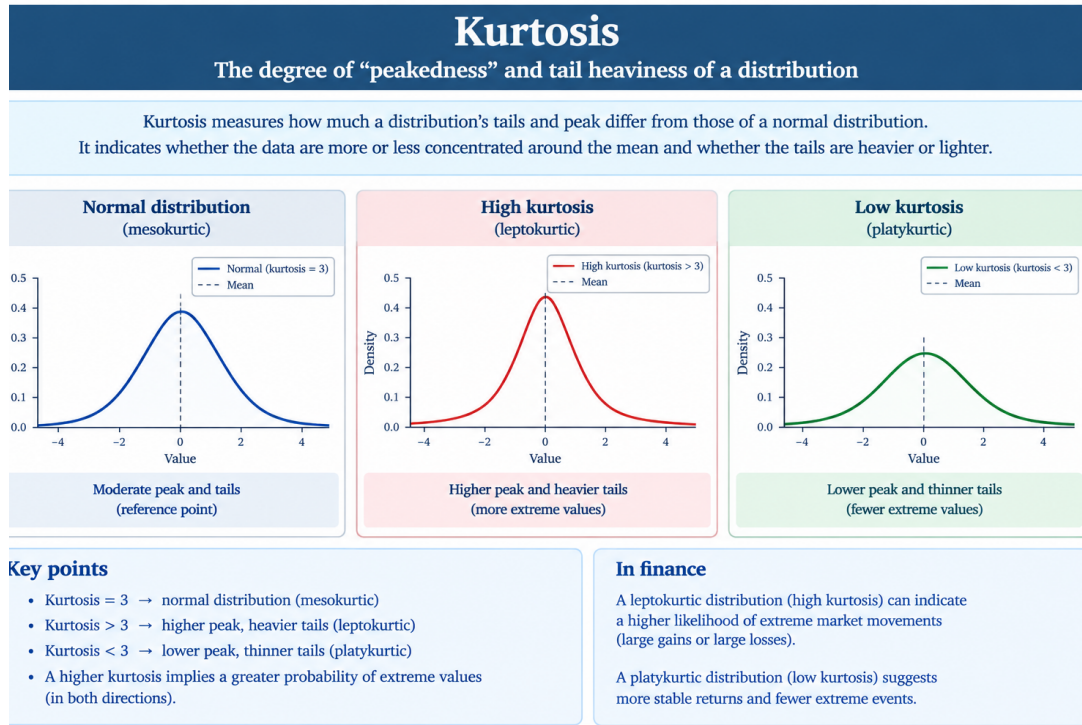


FIGURE 2.2 – Illustration of a negatively skewed distribution.

Difference Between Skewness and Kurtosis It is important to distinguish skewness from kurtosis.

Skewness primarily measures the asymmetry of the distribution :

$$\text{Skewness}(r) < 0 \implies \text{asymmetry toward negative returns} \quad (2.8)$$

Kurtosis primarily measures the importance of the distribution tails and the presence of extreme observations :

$$K_F > 0 \implies \text{fatter tails and more extreme values} \quad (2.9)$$

We can therefore remember :

$$\text{Skewness} \longrightarrow \text{distribution asymmetry} \quad (2.10)$$

and :

$$\text{Kurtosis} \longrightarrow \text{importance of extreme events} \quad (2.11)$$

An asset can therefore simultaneously exhibit negative skewness and positive kurtosis :

$$\text{Skewness}(r) < 0 \quad \text{and} \quad K_F > 0 \quad (2.12)$$

In this case, the return distribution exhibits both **asymmetry toward losses** and **fat tails**.

This combination is particularly important in risk management because it indicates that extreme losses can be both asymmetric and more frequent than suggested by a normal distribution.

2.3 The Jarque-Bera normality test :

Normality test based on the formula $JB = \frac{T}{6} \left(S^2 + \frac{K_{ex}^2}{4} \right)$, where :

- T represents the number of observations ;
- S represents skewness ;
- K represents excess kurtosis.

The statistic therefore combines the deviation from zero skewness and the deviation from zero excess kurtosis.

The function `stats.jarque_bera(.)` returns two values : the test statistic and the *p-value*. If the *p-value* < 0.05 , the normality hypothesis is rejected at the 95% confidence level.

Explanation : The **Jarque-Bera** test determines whether a data series is compatible with a **normal distribution**. It is particularly useful in finance for checking whether returns can reasonably be modeled using a normal distribution.

The test is based on two characteristics of the distribution : **skewness** and **kurtosis**.

Recall that for a normal distribution : Skewness = 0 and and, when using Fisher's excess kurtosis : Kurtosis = 0, so that $JB = 0$.

Test Hypotheses The null hypothesis is :

$$H_0 : \text{the distribution of the data is normal} \quad (2.13)$$

The alternative hypothesis is :

$$H_1 : \text{the distribution of the data is not normal} \quad (2.14)$$

Interpretation of the p-value A significance level of 5% is generally used :

$$\alpha = 0.05 \quad (2.15)$$

If :

$$p\text{-value} < 0.05 \quad (2.16)$$

then H_0 is rejected and we conclude that the data exhibit characteristics significantly different from those of a normal distribution.

Conversely, if :

$$p\text{-value} > 0.05 \quad (2.17)$$

we do not reject H_0 . This means that the test does not provide sufficient evidence to reject the normality hypothesis.

p-value of the Jarque–Bera test Under the null hypothesis of normality, and asymptotically,

$$JB \stackrel{a}{\sim} \chi^2(2)$$

that is, the Jarque–Bera statistic asymptotically follows a χ^2 distribution with two degrees of freedom. The p-value then corresponds to the probability, under the null hypothesis, of observing a test statistic at least as large as the one obtained from the sample :

$$P(\chi^2_2 \geq JB_{\text{obs}})$$

or, equivalently, using the cumulative distribution function $F_{\chi^2_2}$ of the χ^2 distribution with two degrees of freedom :

$$1 - F_{\chi^2_2}(JB_{\text{obs}})$$

In Python, the test can be performed using :

```
jb_stat, p_value = stats.jarque_bera(log_returns_filtered['BTC-USD'])
```

```

1 import yfinance as yf
2 import pandas as pd
3 import numpy as np
4 import scipy.stats as stats
5
6 # Data ingestion for a highly volatile asset (e.g., Bitcoin)
7 ticker = 'BTC-USD'
8 prices = yf.download(ticker, start="2021-01-01", end="2026-01-01")['Adj Close'].
          ffill()
9 log_returns = np.log(prices / prices.shift(1)).dropna()
10
11 # Calculation of distribution moments via scipy.stats
12 skewness_val = stats.skew(log_returns)
13 kurtosis_ex = stats.kurtosis(log_returns, fisher=True)
14
15 # Execution of the Jarque-Bera test
16 jb_stat, p_value = stats.jarque_bera(log_returns)
17
18 print(f"--- Statistical Analysis of {ticker} ---")
19 print(f"Skewness : {skewness_val:.4f} (Expected for a normal distribution: 0)")
20 print(f"Excess Kurtosis : {kurtosis_ex:.4f} (Expected for a normal
      distribution: 0)")
21 print(f"Jarque-Bera Statistic : {jb_stat:.2f}")
22 print(f"Test p-value : {p_value:.6f}")
23
24 if p_value < 0.05:
25     print("Conclusion: We reject H0. The distribution is non-Gaussian.")
26 else:
27     print("Conclusion: We cannot reject H0.")

```

Listing 2.1 – Normality Test and Higher-Order Moment Analysis

2.4 The Shapiro–Wilk Normality Test

The Shapiro–Wilk test is another statistical test used to determine whether a sample is compatible with a normal distribution.

In the financial context, this test can be used to assess whether the observed returns of an asset or portfolio can reasonably be considered normally distributed.

Test hypotheses Consider a sample of returns :

$$X_1, X_2, \dots, X_n.$$

The Shapiro–Wilk test is based on the following hypotheses :

$$\begin{array}{l} H_0 : X \sim \mathcal{N}(\mu, \sigma^2), \\ H_1 : X \text{ does not follow a normal distribution.} \end{array}$$

The null hypothesis H_0 therefore means that the observations are compatible with a normal distribution. The objective of the test is to determine whether the data provide sufficient evidence to reject this hypothesis.

General principle of the test The idea behind the Shapiro–Wilk test is to compare the observations in the sample with what we would expect if the observations actually came from a normal distribution.

The first step consists of ordering the observations in ascending order :

$$X_{(1)} \leq X_{(2)} \leq \dots \leq X_{(n)}.$$

The $X_{(i)}$ are called the **order statistics**. For example, for the sample :

$$\{0.03, -0.01, 0.02, 0.05, -0.02\},$$

we obtain :

$$X_{(1)} = -0.02, \quad X_{(2)} = -0.01, \quad X_{(3)} = 0.02, \quad X_{(4)} = 0.03, \quad X_{(5)} = 0.05.$$

The test then compares this ordered structure with what would be observed in a sample drawn from a normal distribution.

The Shapiro–Wilk statistic The test statistic is given by :

$$W = \frac{\left(\sum_{i=1}^n a_i X_{(i)} \right)^2}{\sum_{i=1}^n (X_i - \bar{X})^2}$$

where :

- $X_{(i)}$ denotes the i -th observation after ordering ;
- $\bar{X}$ is the sample mean ;
- a_i are the coefficients of the Shapiro–Wilk test. They are chosen to efficiently measure the deviation between the observed distribution and a normal distribution.
- n is the sample size.

The statistic W generally lies between 0 and 1.

- A value of W close to 1 indicates that the structure of the data is compatible with what is expected under a normal distribution.
- Conversely, a lower value of W indicates a greater deviation from normality.
- However, the final decision of the test is generally based on the **p-value**, rather than simply on whether W is close to 1.

p-value of the Shapiro–Wilk test Once the observed statistic W_{obs} has been calculated, the p-value corresponds to the probability, under the null hypothesis, of obtaining a value of W at least as small as the one observed :

$$P(W \leq W_{\text{obs}} \mid H_0)$$

The direction of the inequality is important. In the Shapiro–Wilk test, values of W close to 1 are associated with data that are more consistent with a normal distribution, whereas low values of W indicate a deviation from normality. Unlike the Jarque–Bera test, the statistic W does not follow a simple standard distribution such as a normal distribution or a χ^2 distribution. The distribution of W under the null hypothesis depends notably on the sample size and is mathematically complex because it does not have a simple analytical form. Therefore, the p-value is obtained using numerical approximations implemented in statistical software.

Where do the coefficients a_i come from? The coefficients a_i are the most technical part of the test. The idea is to construct coefficients that allow us to best compare the ordered observations in the sample with what we would expect to observe under a normal distribution. To understand their origin, consider a sample of independent random variables following a standard normal distribution :

$$Z_1, Z_2, \dots, Z_n \sim \mathcal{N}(0, 1).$$

These observations are also ordered :

$$Z_{(1)} \leq Z_{(2)} \leq \dots \leq Z_{(n)}.$$

We then consider the expected values of the order statistics :

$$m_i = E[Z_{(i)}].$$

The vector of expected values is therefore :

$$m = \begin{pmatrix} m_1 \\ m_2 \\ \vdots \\ m_n \end{pmatrix}.$$

This vector represents the average positions that the ordered observations should have if the data came from a standard normal distribution. For example : $m_1 < 0$, $m_n > 0$. Moreover, due to the symmetry of the normal distribution : $m_i = -m_{n+1-i}$.

The covariance matrix of the order statistics The order statistics $Z_{(1)}, Z_{(2)}, \dots, Z_{(n)}$ are not independent. Therefore, we define their covariance matrix as :

$$V = \text{Cov} \begin{pmatrix} Z_{(1)} \\ Z_{(2)} \\ \vdots \\ Z_{(n)} \end{pmatrix}.$$

In other words : $V_{ij} = \text{Cov}(Z_{(i)}, Z_{(j)})$.

Based on the vector m and the matrix V , the coefficients a_i are constructed in order to assign appropriate weights to the different ordered observations. A theoretical expression, depending on the convention used for the vectors, is :

$$a = \frac{m^\top V^{-1}}{\sqrt{m^\top V^{-1} V^{-1} m}}.$$

This construction determines the weights that make the comparison between the observations and the structure expected under normality as effective as possible.

Why are the a_i not calculated by hand? In practice, the coefficients a_i are generally not calculated directly by hand. Indeed, one would first need to determine $E[Z_{(i)}]$ as well as the covariances $\text{Cov}(Z_{(i)}, Z_{(j)})$. These calculations quickly become complex as n increases.

Statistical software therefore uses numerical methods and specific approximations to obtain the coefficients required for the test.

For example, with Python and the `scipy` library, we can simply use : `scipy.stats.shapiro()`.

The software automatically calculates the statistic W as well as the p-value.

Therefore, it is mainly important to understand the role of the coefficients a_i , rather than trying to calculate them manually. We can summarize this as follows :

The a_i weight the ordered observations in order to compare their structure with that expected under a normal distribution.

Intuitively :

$$W = \frac{\text{structure compatible with normality}}{\text{total variability}}.$$

- When the data are close to what is expected under normality, W is close to 1.
- When the data exhibit a substantial deviation from normality, W tends to decrease.

Statistical decision Once the statistic W has been calculated, the test provides a p-value. For a significance level of : $\alpha = 5\% = 0.05$, we apply the following rule :

$$\begin{cases} p\text{-value} < 0.05 & \Rightarrow \text{reject } H_0, \\ p\text{-value} \geq 0.05 & \Rightarrow \text{do not reject } H_0. \end{cases}$$

Thus, $p < 0.05$ means that the data provide sufficient evidence to conclude that they are not compatible with a normal distribution at the 5% significance level.

Conversely, $p \geq 0.05$ means that there is not sufficient evidence to reject normality.

Important : failing to reject H_0 does not mean that we have proved that the data are normally distributed. The correct formulation is :

“We do not reject the normality hypothesis.”

and not :

“The data are normally distributed.”

Python implementation The `scipy.stats` library allows us to directly perform the Shapiro–Wilk test.

Shapiro–Wilk test

```

1 import numpy as np
2 from scipy.stats import shapiro
3
4 # Example of returns
5 returns = np.array([
6     0.012, -0.008, 0.005, 0.003, -0.011,
7     0.007, -0.004, 0.009, -0.002, 0.006
8 ])
9
10 # Shapiro-Wilk test
11 W, p_value = shapiro(returns)
12
13 print("Shapiro-Wilk statistic:", W)
14 print("p-value:", p_value)
15
16 # Significance level
17 alpha = 0.05
18
19 if p_value < alpha:
20     print("Reject H0: the returns are not normally distributed.")
21 else:
22     print("Do not reject H0: the returns are consistent with normality.")

```

The result contains two main pieces of information :

- W : the test statistic ;
- the p-value : used to make the statistical decision.

Comparison between Shapiro–Wilk and Jarque–Bera Both tests have the same null hypothesis :

$$H_0 : X \sim \mathcal{N}(\mu, \sigma^2).$$

However, they do not test normality in exactly the same way.

	Jarque–Bera	Shapiro–Wilk
Null hypothesis	The data follow a normal distribution	The data follow a normal distribution
Statistic	JB	W
Main idea	Uses skewness and kurtosis	Compares the structure of the ordered observations with that expected under normality
Characteristic value	$S = 0$, $K = 3$ under normality	W close to 1 under normality
Decision	Based on the p-value	Based on the p-value
If $p < 0.05$	Reject normality	Reject normality

TABLE 2.1 – Comparison between the Jarque–Bera and Shapiro–Wilk tests.

Joint interpretation of the two tests It can be useful to apply both tests to the same returns. If $p_{SW} > 0.05$ and $p_{JB} > 0.05$, neither test provides sufficient evidence to reject the normality hypothesis at the 5% significance level.

We can therefore conclude :

Normality is not rejected by either test at the 5% significance level.

Conversely, if $p_{SW} < 0.05$ and $p_{JB} < 0.05$, both tests reject the normality hypothesis. It is then reasonable to conclude that the returns exhibit a statistically significant deviation from a normal distribution.

When the two tests lead to different conclusions It is possible that the two tests do not lead to the same conclusion. For example, $p_{SW} < 0.05$ but $p_{JB} > 0.05$. This means that Shapiro–Wilk rejects normality whereas Jarque–Bera does not reject it.

This difference is not necessarily contradictory because the two tests use different information :

Shapiro–Wilk $\longrightarrow$ overall structure of the ordered observations

whereas :

Jarque–Bera $\longrightarrow$ skewness + kurtosis.

A difference in conclusions can therefore occur when the deviation from normality comes from a particular characteristic of the distribution, for example :

- substantial skewness ;
- heavier tails than those of a normal distribution ;
- extreme observations ;
- another form of deviation from normality.

$\Rightarrow$ The two tests are therefore complementary rather than competing.

Remark For financial returns, normality is an important assumption in many classical models. However, financial returns frequently exhibit skewness, kurtosis greater than 3, and fat tails. It is therefore particularly useful to combine several tools in order to obtain a more complete view of the distribution of returns.

2.5 Regulatory Risk Measures : VaR and CVaR

In financial risk management, knowing the average return of a portfolio is not sufficient : it is also essential to quantify the magnitude of potential losses, particularly under adverse and extreme scenarios. **Value-at-Risk (VaR)** and **Conditional Value-at-Risk (CVaR)**, also known as *Expected Shortfall (ES)*, are two fundamental measures used to quantify this risk.

VaR answers the following question :

What level of loss should the portfolio exceed only with a given probability, over a given horizon ?

Concrete Example. Suppose a portfolio has a **95% VaR over a one-day horizon** of 50,000.

The term “1 day” means that we study the portfolio’s loss over a single day. The 95% confidence level means that we consider the quantile corresponding to the worst 5% of scenarios.

Mathematically, if $L_{t,1j}$ denotes the portfolio loss over one day, then :

$$\mathbb{P}(L_{t,1j} > 50,000) = 5\% \quad (2.18)$$

or equivalently :

$$\mathbb{P}(L_{t,1j} \leq 50,000) = 95\% \quad (2.19)$$

In other words, for a given day, the model estimates a 5% probability that the loss exceeds 50,000. It is important to distinguish the different elements of this definition :

$\underbrace{1 \text{ day}}_{\text{horizon}}$	$\underbrace{95\%}_{\text{confidence level}}$	$\underbrace{5\%}_{\text{exceedance probability}}$
---	---	--

(2.20)

VaR therefore only provides a **loss threshold**. It does not specify the magnitude of the loss once this threshold is exceeded. This is precisely the role of **CVaR**. CVaR measures the **average loss in scenarios where VaR is exceeded**. It therefore answers the following question :

When the loss exceeds the VaR threshold, what is the average magnitude of that loss ?

For example, if the one-day 95% VaR is 50,000 and the one-day 95% CVaR is 75,000, this means that among the scenarios belonging to the worst 5% loss tail, the average loss is 75,000.

CVaR therefore focuses on the portion of the distribution beyond the VaR threshold. Thus, the two measures provide complementary information :

$$\text{VaR} \longrightarrow \text{threshold separating ordinary scenarios from extreme scenarios} \quad (2.21)$$

$$\text{CVaR} \longrightarrow \text{average loss in extreme scenarios beyond VaR} \quad (2.22)$$

This distinction is particularly important in quantitative finance. Two portfolios may have the same VaR while having very different CVaRs. The portfolio with the higher CVaR is then exposed to potentially larger extreme losses.

Definition 2.5.1 *In general, consider a random variable L representing the **loss** of a portfolio over a given time horizon. For a confidence level $\alpha \in (0,1)$,*

— *the **Value-at-Risk** is defined as the α -quantile of the loss distribution :*

$$P(L > \text{VaR}_\alpha) = \alpha \quad (2.23)$$

— **Expected Shortfall (CVaR)** : *which measures the expected loss (absolute value), conditional on the loss exceeding the VaR threshold, is defined by*

$$\text{CVaR}_\alpha = E[L \mid L > \text{VaR}_\alpha] \quad (2.24)$$

Python Toolbox for Risk Calculation : To implement our three risk models (Historical, Parametric, Cornish-Fisher), we use the following key functions :

— `np.percentile(df, q)` : Extracts the q -th percentile of the distribution. For a historical VaR with a risk threshold $\alpha = 5\%$, we seek to isolate the breakpoint corresponding to the worst 5% of returns in the time series.

- `stats.norm.ppf(q)` : *Percent Point Function* (or quantile function). It is the inverse of the cumulative distribution function of the normal distribution. For $\alpha = 0.05$, it returns the standardized score $z_\alpha \approx -1.6448$.
- `df[df <= threshold].mean()` : Uses Pandas Boolean masking to isolate only returns less than or equal to the VaR quantile, and then computes their average. This is the direct implementation of the conditional expectation of historical Expected Shortfall.

```

1 # Definition of risk parameters
2 alpha = 0.05 # 5% threshold (95% confidence)
3 capital = 1000000 # $1,000,000 portfolio
4
5 # Calculation of empirical parameters
6 mu = log_returns.mean()
7 sigma = log_returns.std()
8
9 # --- METHOD 1: HISTORICAL (NON-PARAMETRIC) ---
10 # Extraction of the 5th empirical percentile
11 var_hist_ret = np.percentile(log_returns, alpha * 100)
12 var_historical = -var_hist_ret * capital
13
14 # Historical CVaR: average of returns below the VaR
15 cvar_hist_ret = log_returns[log_returns <= var_hist_ret].mean()
16 cvar_historical = -cvar_hist_ret * capital
17
18 # --- METHOD 2: GAUSSIAN PARAMETRIC ---
19 # Retrieval of the classical z-score via stats.norm.ppf
20 z_alpha = stats.norm.ppf(alpha)
21 var_parametric = -(mu + z_alpha * sigma) * capital
22
23 # Gaussian CVaR:
24 # Theoretical average of returns located in the left tail
25 # For alpha = 0.05, phi(z_alpha) / alpha corresponds to the expectation
26 # of a standard normal variable conditioned on Z <= z_alpha
27 cvar_parametric_ret = -(mu - sigma * stats.norm.pdf(z_alpha) / alpha)
28 cvar_parametric = cvar_parametric_ret * capital
29
30 # --- METHOD 3: CORNISH-FISHER (NON-GAUSSIAN ADJUSTMENT) ---
31 S = stats.skew(log_returns)
32 K = stats.kurtosis(log_returns, fisher=True)
33
34 # Polynomial extension of the z_alpha
35 z_cf = (z_alpha +
36         (z_alpha**2 - 1) * S / 6 +
37         (z_alpha**3 - 3 * z_alpha) * K / 24 -
38         (2 * z_alpha**3 - 5 * z_alpha) * (S**2) / 36)
39
40 var_cornish_fisher = -(mu + z_cf * sigma) * capital
41
42 # Cornish-Fisher CVaR:
43 # Cornish-Fisher directly provides an approximation of the quantile.
44 # To obtain an approximation of CVaR, we integrate the Cornish-Fisher
45 # quantiles over the entire left tail of the distribution.
46 u_values = np.linspace(0.0001, alpha, 1000)
47
48 z_values = stats.norm.ppf(u_values)
49
50 z_cf_values = (z_values +
51               (z_values**2 - 1) * S / 6 +
52               (z_values**3 - 3 * z_values) * K / 24 -
53               (2 * z_values**3 - 5 * z_values) * (S**2) / 36)

```

```

54
55 q_cf_values = mu + sigma * z_cf_values
56
57 # Numerical integration of the left-tail quantiles
58 cvar_cf_ret = np.trapezoid(q_cf_values, u_values) / alpha
59 cvar_cornish_fisher = -cvar_cf_ret * capital
60
61 print("\n--- Risk Analysis (1-Day Horizon, $1M Capital) ---")
62 print(f"Historical VaR (95\%)           : {var_historical:.2f} $")
63 print(f"Historical CVaR (95\%)          : {cvar_historical:.2f} $")
64 print(f"Gaussian VaR (95\%)              : {var_parametric:.2f} $")
65 print(f"Gaussian CVaR (95\%)             : {cvar_parametric:.2f} $")
66 print(f"Cornish-Fisher VaR (95\%)        : {var_cornish_fisher:.2f} $")
67 print(f"Cornish-Fisher CVaR (95\%)       : {cvar_cornish_fisher:.2f} $")

```

Listing 2.2 – Comparative Implementation of VaR and CVaR

Mathematical Origin of the Cornish-Fisher Expansion

The **Cornish-Fisher** formula is derived from the **Edgeworth expansion**, which approximates the cumulative distribution function of a non-normal distribution using the normal distribution. We seek the quantile q_α satisfying :

$F(q_\alpha) = \alpha$, where F is the cumulative distribution function of a standard normal distribution

We then express this quantile as a correction of the normal quantile z_α :

$$q_\alpha = z_\alpha + \delta,$$

where δ represents a correction related to the non-normal characteristics of the distribution.

By expanding the Edgeworth expansion around z_α and then approximately inverting the relationship, we obtain the Cornish-Fisher expansion :

$$q_\alpha \approx z_\alpha + \frac{S}{6}(z_\alpha^2 - 1) + \frac{K}{24}(z_\alpha^3 - 3z_\alpha) - \frac{S^2}{36}(2z_\alpha^3 - 5z_\alpha) \quad (2.25)$$

where :

$$z_\alpha = \text{normal distribution quantile}, \quad S = \text{skewness}, \quad K = \text{excess kurtosis}. \quad (2.26)$$

The formula therefore corrects the normal quantile z_α to account for the **asymmetry** of the distribution, represented by S , and the **thickness of its tails**, represented by K .

When the distribution is normal :

$$S = 0 \quad \text{and} \quad K = 0. \quad (2.27)$$

The formula then reduces to :

$$q_\alpha = z_\alpha \quad (2.28)$$

The terms $z_\alpha^2 - 1$ and $z_\alpha^3 - 3z_\alpha$ naturally appear in the Edgeworth expansion and are related to the **Hermite polynomials** associated with the normal distribution.

2.5.1 Mathematical Interpretation of Cornish-Fisher CVaR

The important difference between VaR and CVaR is that the Cornish-Fisher formula directly provides an approximation of a **quantile**, and therefore of VaR, but not directly of CVaR. The Cornish-Fisher VaR is approximately given by :

$$\boxed{\text{VaR}_\alpha^{CF} \approx -q_\alpha^{CF}} \quad (2.29)$$

where q_α^{CF} represents the return quantile obtained using the Cornish-Fisher expansion. CVaR, however, requires considering all quantiles located in the left tail of the distribution. It corresponds to the average loss associated with the most unfavorable scenarios. For a variable representing returns, CVaR can therefore be approximated by :

$$\boxed{\text{CVaR}_\alpha^{CF} \approx -\frac{1}{\alpha} \int_0^\alpha q_u^{CF} du} \quad (2.30)$$

where q_u^{CF} denotes the Cornish-Fisher quantile corresponding to the probability level u . In our example, with $\alpha = 0.05$, we therefore consider the quantiles between 0 and 5% :

$$\text{CVaR}_{0.05}^{CF} \approx -\frac{1}{0.05} \int_0^{0.05} q_u^{CF} du. \quad (2.31)$$

The idea therefore differs from VaR. VaR uses a single quantile :

$$\text{VaR}_\alpha^{CF} \approx -q_\alpha^{CF}, \quad (2.32)$$

whereas CVaR uses all the quantiles in the tail :

$$\text{CVaR}_\alpha^{CF} \approx -\frac{1}{\alpha} \int_0^\alpha q_u^{CF} du. \quad (2.33)$$

In Python, the integral is approximated numerically by constructing a grid of probabilities :

$$u_1, u_2, \dots, u_M \in [0, \alpha]. \quad (2.34)$$

We then calculate the corresponding Cornish-Fisher quantile for each value u_i :

$$q_{u_i}^{CF} = \mu + \sigma z_{CF}(u_i). \quad (2.35)$$

Finally, the integral is approximated numerically using the trapezoidal rule :

$$\int_0^\alpha q_u^{CF} du \approx \text{Trapz}(q_{u_1}^{CF}, \dots, q_{u_M}^{CF}). \quad (2.36)$$

This provides an approximation of CVaR that accounts for the skewness and excess kurtosis of returns through the Cornish-Fisher expansion.

Thus :

$$\boxed{\text{Cornish-Fisher} \rightarrow \text{corrected quantiles} \rightarrow \text{tail integration} \rightarrow \text{CVaR}} \quad (2.37)$$

Chapitre 3

Modern Portfolio Theory & Optimization

3.0.1 Introduction to the Markowitz Framework

Published in 1952 by Harry Markowitz, *Modern Portfolio Theory* (MPT) laid the scientific foundations of diversification. Markowitz's fundamental paradigm is that a financial asset should not be analyzed in isolation, but according to its contribution to the overall risk-return trade-off of the portfolio to which it belongs.

The investor seeks to construct an optimal portfolio by combining assets whose imperfect correlation makes it possible to reduce overall risk (variance) without proportionally sacrificing expected return.

The Mathematical Framework Let us consider an investment universe composed of N risky financial assets. We define the vector of weights W allocated to each asset as :

$$W = \begin{pmatrix} w_1 \\ w_2 \\ \vdots \\ w_N \end{pmatrix}$$

For a portfolio with no short selling and fully invested, we impose the following structural constraints :

$$\sum_{i=1}^N w_i = 1 \quad \text{and} \quad w_i \geq 0 \quad \forall i \in \{1, \dots, N\}$$

Let $\mu_i = E(R_i)$ be the expected return of asset i , and μ the column vector of these expected returns. The expected return of the overall portfolio, denoted $E(R_p)$, is the mathematical expectation of the weighted sum of returns :

$$E(R_p) = \sum_{i=1}^N w_i E(R_i) = W^T \mu$$

Let $\Sigma \in \mathbb{R}^{N \times N}$ be the variance-covariance matrix of asset returns. The generic element σ_{ij} represents the covariance between asset i and asset j (if $i = j$, $\sigma_{ii} = \sigma_i^2$ is the variance of asset i). The overall portfolio variance σ_p^2 can be expressed as a quadratic form :

$$\sigma_p^2 = \sum_{i=1}^N \sum_{j=1}^N w_i w_j \sigma_{ij} = W^T \Sigma W$$

Thus, the portfolio standard deviation (or volatility) is

$$\sigma_p = \sqrt{W^T \Sigma W}$$

The objective of portfolio optimization is to determine how to allocate capital among several assets in order to obtain the best possible trade-off between return and risk.

Two classical formulations can be used to approach this problem. The first consists of finding, for a given level of return, the portfolio with the minimum risk. The second consists of directly finding the portfolio that maximizes the excess return obtained per unit of risk.

These two approaches are closely related and make it possible to understand the construction of the efficient frontier and the identification of the tangency portfolio.

3.0.2 Formulation 1 : Risk Minimization under a Target Return Constraint

A first way to construct the efficient frontier is to fix a target expected return level μ_0 and search, among all portfolios capable of achieving this return, for the one with the lowest risk.

Let

$$W = \begin{pmatrix} w_1 \\ w_2 \\ \vdots \\ w_n \end{pmatrix}$$

be the vector of the weights of the different assets, μ the vector of expected returns, and Σ the covariance matrix of returns.

The optimization problem therefore consists of minimizing this variance for a fixed target return μ_0 :

$$\begin{array}{ll} \min_W & \frac{1}{2} W^T \Sigma W \\ (1) \quad \text{subject to :} & W^T \mu = \mu_0, \\ & W^T \mathbf{1} = 1, \\ & W \geq 0. \end{array}$$

The factor $\frac{1}{2}$ does not modify the optimal solution. It is traditionally introduced in order to simplify the derivatives of the objective function.

The different constraints have a simple economic interpretation :

- $W^T \mu = \mu_0$ imposes that the portfolio achieves exactly the target expected return ;
- $W^T \mathbf{1} = 1$ imposes that the entire capital is invested ;
- $W \geq 0$ prohibits short selling : each weight must be positive or zero.

For a given value of μ_0 , the optimization therefore provides the portfolio with **minimum variance** among all portfolios with the target return.

By progressively varying the target return μ_0 , we obtain a sequence of optimal portfolios. Each portfolio is characterized by a pair :

$$(\sigma_p, \mu_p).$$

The representation of these pairs in the return-risk plane then makes it possible to construct the **efficient frontier**.

Graphical Interpretation The following graph schematically illustrates the construction of the efficient frontier.

The region represents the set of feasible portfolios. Each portfolio is characterized by its risk σ_p and expected return μ_p . For a target return μ_0 , we search for the feasible portfolio with the lowest risk.

The resulting point belongs to the efficient frontier. By repeating this optimization for different target return levels, we progressively construct the efficient frontier.

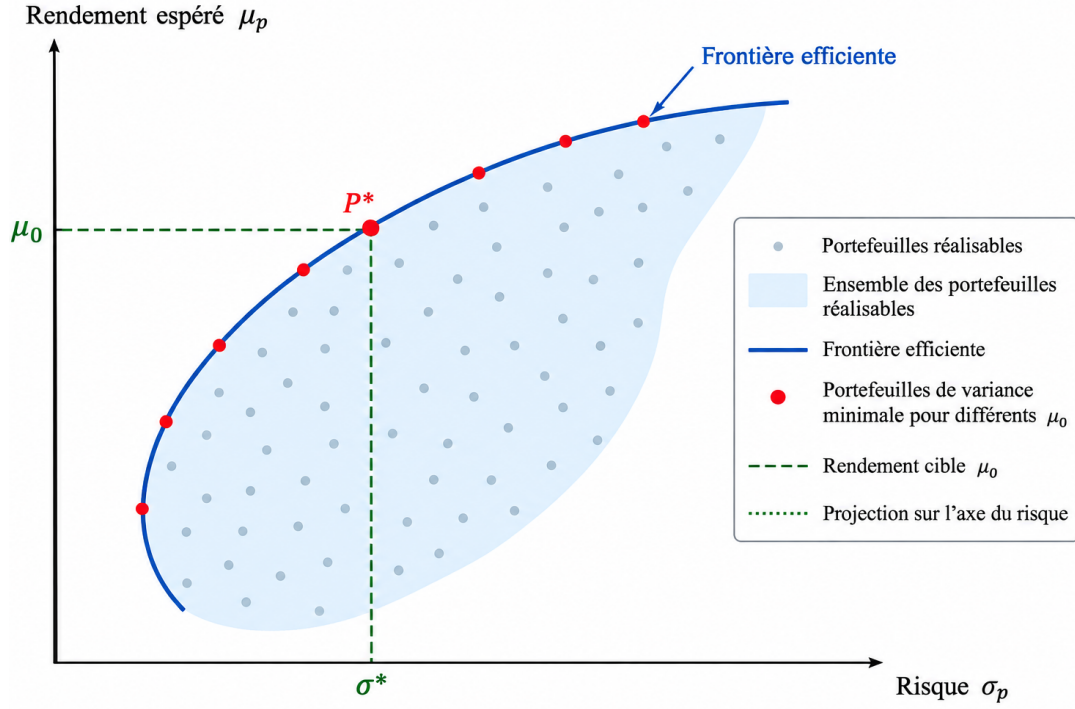


FIGURE 3.1 – Illustration of a negatively skewed distribution.

The principle is therefore as follows :

$$\boxed{\text{Target return} \longrightarrow \text{Variance minimization} \longrightarrow \text{Optimal portfolio}}$$

By repeating this optimization for different values of μ_0 , we obtain the efficient frontier.

3.0.3 Formulation 2 : Sharpe Ratio Maximization

When a risk-free asset offering a certain return R_f is introduced, it becomes possible to compare risky portfolios while taking into account both their return and their level of risk.

The **Sharpe Ratio** measures the excess return obtained per unit of risk :

$$\boxed{SR(W) = \frac{W^T \mu - R_f}{\sqrt{W^T \Sigma W}}}$$

where :

- $W^T \mu$ represents the expected return of the risky portfolio ;
- R_f represents the return of the risk-free asset ;
- $\sqrt{W^T \Sigma W} = \sigma_p$ represents the volatility of the portfolio.

The objective is then to find the risky portfolio that maximizes the Sharpe Ratio :

$$\begin{array}{l} \max_W \quad \frac{W^T \mu - R_f}{\sqrt{W^T \Sigma W}} \\ \text{subject to : } \quad W^T \mathbf{1} = 1, \\ \quad \quad \quad W \geq 0. \end{array}$$

In practice, many numerical optimization algorithms, including those available in `scipy.optimize`, are formulated as minimization problems.

It is therefore sufficient to change the sign of the objective function :

$$\min_W \quad -SR(W) = -\frac{W^T \mu - R_f}{\sqrt{W^T \Sigma W}}$$

subject to the same constraints :

$$W^T \mathbf{1} = 1, \quad W \geq 0.$$

The Tangency Portfolio : The solution to this problem is called the **tangency portfolio**, or **maximum Sharpe portfolio**.

To understand its graphical importance, let us place the risk-free asset in the return-risk plane. A risk-free asset has zero volatility. It is therefore represented by the point $(0, R_f)$.

Now consider a risky portfolio P , characterized by (σ_P, μ_P) . The line connecting the risk-free asset to portfolio P has slope :

$$\frac{\mu_P - R_f}{\sigma_P}.$$

But this expression corresponds exactly to the Sharpe Ratio of the portfolio :

$$\text{slope of the line} = SR(P)$$

Thus, maximizing the Sharpe Ratio graphically means searching, among all lines originating from the risk-free asset, for the one with the highest slope.

The portfolio corresponding to the point of tangency with the efficient frontier is called the **tangency portfolio**.

The line passing through the risk-free asset and the tangency portfolio is called the **Capital Market Line** (CML). Its slope is equal to the maximum Sharpe Ratio :

$$\text{slope of the CML} = SR(W^*) = \max_W SR(W)$$

where W^* denotes the tangency portfolio.

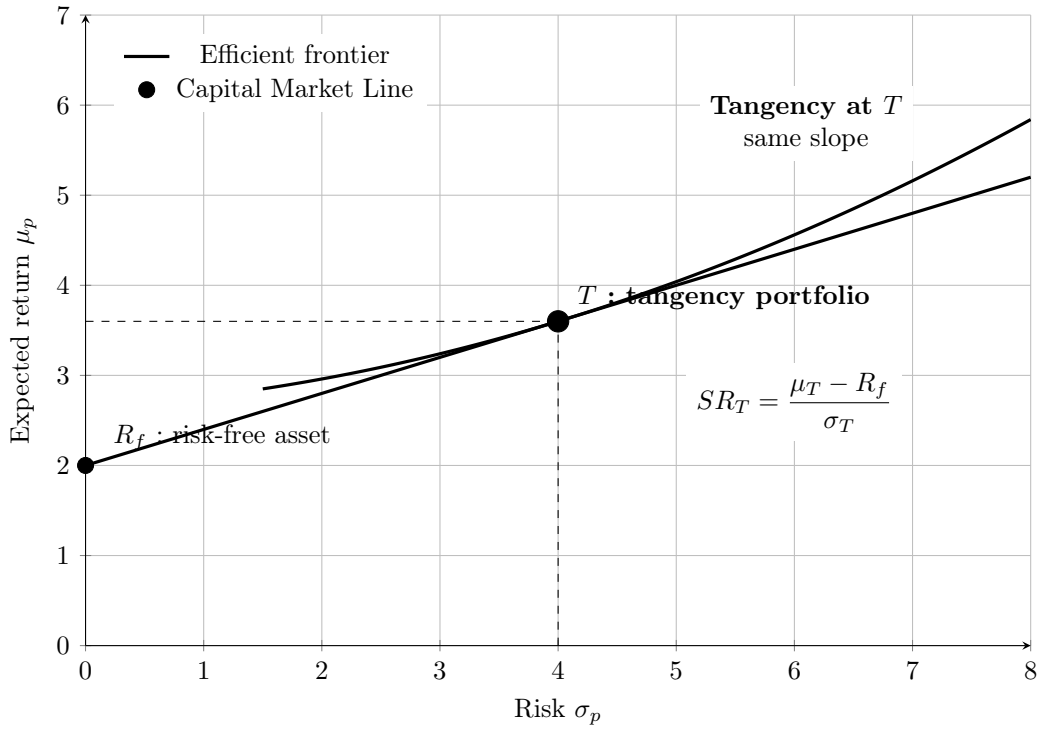


FIGURE 3.2 – The tangency portfolio T is the point where the Capital Market Line is tangent to the efficient frontier. The slope of this line is equal to the Sharpe Ratio of the tangency portfolio. The data are purely illustrative.

Combination of the Risk-Free Asset and the Tangency Portfolio

Once the tangency portfolio W^* has been determined, the investor can combine this portfolio with the risk-free asset. If α ($0 \leq \alpha \leq 1$) denotes the proportion invested in the tangency portfolio, then the proportion invested in the risk-free asset is $1 - \alpha$ and the expected return of the overall portfolio is :

$$\mu_C = (1 - \alpha)R_f + \alpha\mu_{W^*}.$$

We can also write :

$$\boxed{\mu_C = R_f + \alpha(\mu_{W^*} - R_f)}$$

and its risk is :

$$\boxed{\sigma_C = \alpha\sigma_{W^*}.}$$

These two equations show that the portfolios obtained by combining the risk-free asset and the tangency portfolio lie on a straight line in the return-risk plane : the **Capital Market Line**.

3.0.4 Formulation 3 : The Sharpe Ratio along the Efficient Frontier

The two previous formulations can be connected through a third representation that is particularly useful for understanding the role of the tangency portfolio.

For each target return μ_0 , the first optimization provides a minimum-variance portfolio $W(\mu_0)$. We can then calculate its Sharpe Ratio :

$$SR(\mu_0) = \frac{\mu_0 - R_f}{\sigma_p(\mu_0)}$$

with :

$$\sigma_p(\mu_0) = \sqrt{W(\mu_0)^T \Sigma W(\mu_0)}.$$

We therefore obtain a function that associates each target return with the Sharpe Ratio of the corresponding efficient portfolio :

$$\mu_0 \mapsto SR(\mu_0)$$

The tangency portfolio then corresponds to the point at which this function reaches its maximum :

$$\mu_0^* = \operatorname{argmax}_{\mu_0} SR(\mu_0)$$

In other words, the tangency portfolio is the efficient portfolio that offers the best excess return per unit of risk.

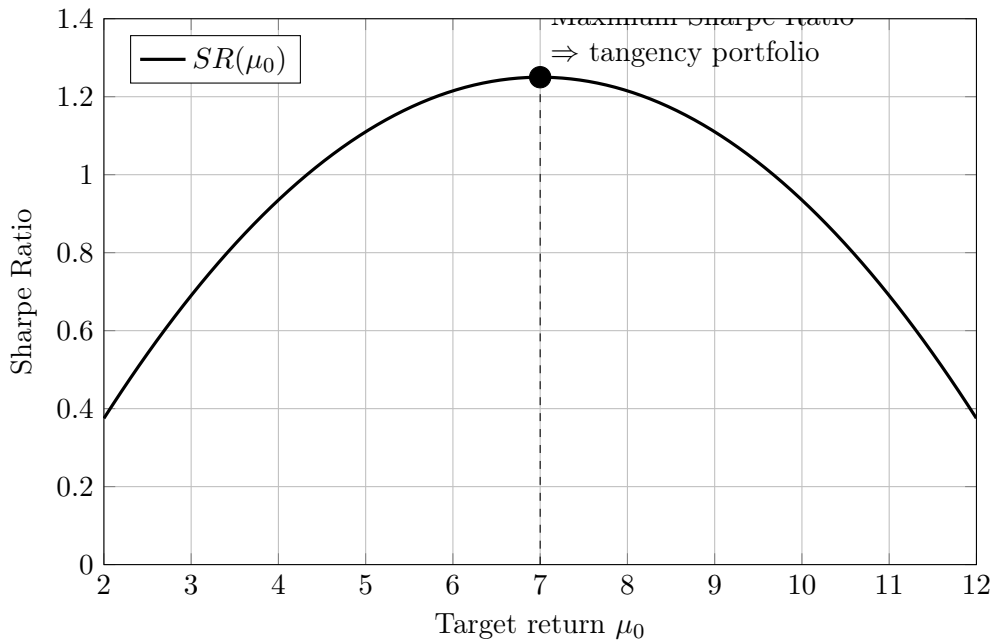


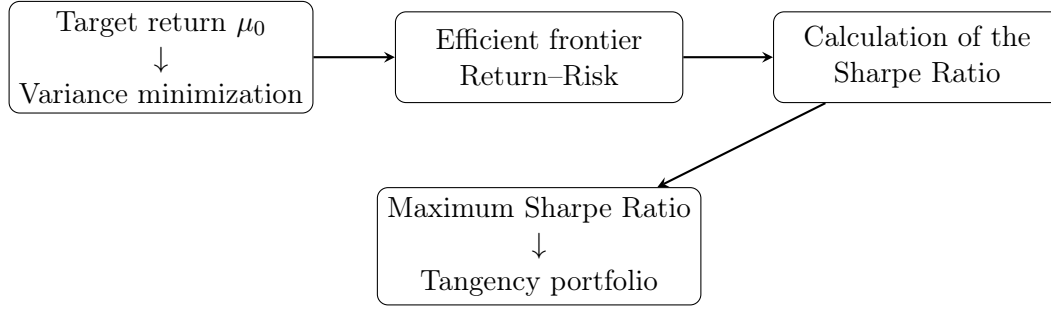
FIGURE 3.3 – Evolution of the Sharpe Ratio along the efficient frontier. The maximum corresponds to the tangency portfolio. The values shown are purely illustrative.

This third graph directly highlights the link between the two optimization problems.

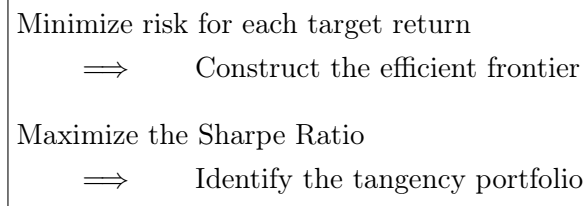
For each value of μ_0 , we have an efficient portfolio. We then calculate its Sharpe Ratio. The maximum of the curve corresponds to the tangency portfolio.

3.0.5 Graphical Summary

The three representations make it possible to progressively understand the reasoning behind portfolio optimization.



The fundamental idea to remember is therefore :



The first formulation therefore makes it possible to construct the efficient frontier, while the second makes it possible to select, on this frontier, the portfolio that offers the best excess return per unit of risk when a risk-free asset is available.

3.0.6 SLSQP Method — Sequential Least Squares Programming

SLSQP is a constrained optimization method based on the principle of *Sequential Quadratic Programming* (SQP).

We seek to solve the problem

$$\min_x f(x) \quad \text{subject to} \quad g(x) = 0, \quad h(x) \geq 0. \quad (3.1)$$

Principle We start from an **initial point** x_0 , chosen by the user. At iteration k , rather than directly searching for the new x , we seek a **displacement** d from the current point :

$$x = x_k + d. \quad (3.2)$$

We then locally approximate the objective using a second-order Taylor expansion :

$$f(x_k + d) \simeq f(x_k) + \nabla f(x_k)^T d + \frac{1}{2} d^T H_k d. \quad (3.3)$$

Since $f(x_k)$ is constant with respect to d , the optimal displacement is obtained by solving :

$$\begin{aligned} \min_d \quad & \nabla f(x_k)^T d + \frac{1}{2} d^T H_k d \\ \text{subject to} \quad & g(x_k) + \nabla g(x_k)^T d = 0, \\ & h(x_k) + \nabla h(x_k)^T d \geq 0. \end{aligned} \quad (3.4)$$

The constraints are here **linearized** around x_k , while the objective is approximated by a quadratic function. The resulting problem is therefore a **quadratic programming (QP) problem**. The solution d_k provides a direction of displacement. The new iterate is then :

$$x_{k+1} = x_k + \alpha_k d_k, \quad (3.5)$$

where α_k is determined by a line search in order to control the length of the displacement.

Role of H_k : H_k represents an **approximation of the Hessian of the Lagrangian**

$$\mathcal{L}(x, \lambda, \mu) = f(x) + \lambda^T g(x) - \mu^T h(x). \quad (3.6)$$

It therefore contains information about the **local curvature** of the problem. In practice, H_0 is often initialized with the identity matrix :

$$H_0 = I, \quad (3.7)$$

and then updated during the iterations using a quasi-Newton method, typically **BFGS**.

Link with Gradient Descent : Gradient descent uses a first-order approximation :

$$f(x_k + d) \simeq f(x_k) + \nabla f(x_k)^T d, \quad (3.8)$$

which leads to the direction :

$$d_k = -\nabla f(x_k). \quad (3.9)$$

SLSQP generalizes this idea to constrained problems : it searches for the **best local displacement** d while taking into account both the **curvature** of the objective through H_k and the **constraints** through their linearization.

Thus :

$$\boxed{\text{Gradient descent : } d_k = -\nabla f(x_k)} \quad (3.10)$$

whereas SLSQP searches for :

$$\boxed{d_k = \arg \min_d \left[\nabla f(x_k)^T d + \frac{1}{2} d^T H_k d \right] \quad \text{subject to linearized constraints.}} \quad (3.11)$$

3.0.7 Some Remarks

In financial *Data Engineering* practice, raw data extracted from markets (via yFinance, for example) are captured at a **daily** frequency. However, performance, management mandates, and asset comparisons are universally expressed over an **annual** horizon. It is therefore essential to annualize the calculated statistics.

A standard trading year contains approximately **252 trading days** (excluding weekends and public holidays).

- Let R_t be the daily log-return of an asset. Under the assumption of stationarity, the expected daily return is $\mu_d = E(R_t)$. The annual log-return is obtained by summing the daily log-returns over the 252 trading days, so the expectation follows by linearity :

$$\mu_{\text{annuel}} = E \left(\sum_{t=1}^{252} R_t \right) = 252 \times E(R_t) = \mu_d \times 252$$

- Under the standard assumption of independent and identically distributed (i.i.d.) daily returns (absence of temporal autocorrelation, a property related to weak-form market efficiency), the variance of a sum of random variables is equal to the sum of their variances :

$$\sigma_{\text{annuel}}^2 = \text{Var} \left(\sum_{t=1}^{252} R_t \right) = \sum_{t=1}^{252} \text{Var}(R_t) = 252 \times \sigma_d^2$$

Therefore, for volatility (standard deviation), the relationship follows a **square-root-of-time** rule :

$$\sigma_{\text{annuel}} = \sqrt{252 \times \sigma_d^2} = \sigma_d \times \sqrt{252}$$

In Python code, directly multiplying the daily variance-covariance matrix `returns.cov()` by 252 makes it possible to annualize all variances (diagonal elements) and covariances (off-diagonal elements) at once according to this statistical rule :

$$\Sigma_{\text{annuel}} = 252 \times \Sigma_{\text{journalière}}.$$

3.0.8 Complete Scripted Implementation

Here is the clean and commented implementation of the complete script in Python :

```

1 # =====
2 # 0. IMPORTING LIBRARIES
3 # =====
4 import numpy as np
5 import pandas as pd
6 import yfinance as yf
7 import matplotlib.pyplot as plt
8 import scipy.optimize as sco
9
10 # =====
11 # 1. DATA EXTRACTION AND CLEANING
12 # =====
13
14 # List of assets composing our investment universe
15 assets = ['AAPL', 'MSFT', 'AMZN', 'NVDA']
16
17 # Retrieval of adjusted closing prices from Yahoo Finance
18 # for the period from 01/01/2021 to 01/01/2026.
19 # Adjusted prices account in particular for splits and dividends.
20 data = yf.download(
21     assets,
22     start='2021-01-01',
23     end='2026-01-01'
24 )['Close']
25
26 # -----
27 # Calculation of daily logarithmic returns
28 # -----
29 # data.shift(1) shifts prices by one day:
30 #
31 #           P_t
32 # r_t = ln( ----- )
33 #           P_{t-1}
34 #
35 # dropna() removes the first observation because there is no
36 # P_{t-1} for the first day.
37 returns = np.log(data / data.shift(1)).dropna()
38
39 # =====
40 # 2. ANNUALIZATION OF STATISTICS
41 # =====
42 # We consider approximately 252 trading days per year.
43 TRADING_DAYS = 252
44
45 # -----
46 # Annualized average return

```

```

47 # -----
48
49 # returns.mean() calculates the average daily logarithmic return
50 # for each asset.
51 # We annualize by multiplying by 252:
52 # mu_i = 252 * average(r_i)
53 mean_returns = returns.mean() * TRADING_DAYS
54
55 # -----
56 # Annualized covariance matrix
57 # -----
58 # returns.cov() calculates the daily covariance matrix:
59 #
60 #               Cov(r_1,r_1)   Cov(r_1,r_2) ...
61 # Sigma_daily = ...
62 # To annualize the covariance:
63 # Sigma = 252 * Sigma_daily
64 #
65 cov_matrix = returns.cov() * TRADING_DAYS
66
67
68 # =====
69 # 3. MONTE CARLO SIMULATION
70 # =====
71
72 # Number of random portfolios we will generate
73 num_portfolios = 10000
74
75 # We create a matrix of dimension 3 x 10,000.
76 # For each portfolio, we will store:
77 # results[0,i] = portfolio volatility
78 # results[1,i] = portfolio return
79 # results[2,i] = Sharpe Ratio
80 #
81 results = np.zeros((3, num_portfolios))
82
83 # This list will store the weights
84 # of each of the 10,000 portfolios.
85 weights_record = []
86
87 # Fixing the random seed to obtain
88 # the same results at each execution.
89 np.random.seed(42)
90
91 # -----
92 # Generation of random portfolios
93 # -----
94
95 for i in range(num_portfolios):
96
97     # Generation of 4 positive random numbers,
98     # one for each of the 4 assets.
99     weights = np.random.random(len(assets))
100
101     # -----
102     # Weight normalization
103     # -----
104     # We impose:
105     #
106     # w_1 + w_2 + ... + w_N = 1
107     # that is, 100% of the capital is invested.

```

```

108 weights /= np.sum(weights)
109
110 # We store the portfolio weights
111 weights_record.append(weights)
112
113 # -----
114 # Portfolio return
115 # -----
116 # Mathematical formula:
117 #  $\mu_p = w^T \mu$ 
118 #
119 # that is:
120 #  $\mu_p = \sum(w_i * \mu_i)$ 
121 #
122 p_ret = np.dot(weights, mean_returns)
123
124 # -----
125 # Portfolio volatility
126 # -----
127 # Portfolio variance:
128 #  $\sigma_p^2 = w^T \Sigma w$ 
129 #
130 # Volatility:
131 #  $\sigma_p = \sqrt{w^T \Sigma w}$ 
132 #
133 p_vol = np.sqrt(
134     np.dot(
135         weights.T,
136         np.dot(cov_matrix, weights)
137     )
138 )
139
140 # -----
141 # Sharpe Ratio
142 # -----
143 # General formula:
144 #
145 #  $\text{Sharpe} = (\mu_p - R_f) / \sigma_p$ 
146 #
147 # Here, we assume  $R_f = 0$ .
148 # Therefore:
149 #  $\text{Sharpe} = \mu_p / \sigma_p$ 
150 #
151 p_sharpe = p_ret / p_vol
152
153 # Storing the results
154 results[0, i] = p_vol
155 results[1, i] = p_ret
156 results[2, i] = p_sharpe
157
158 # =====
159 # 4. NUMERICAL OPTIMIZATION
160 # =====
161 # We define a function that calculates the three statistics
162 # of a portfolio from its weights.
163 def portfolio_stats(weights):
164
165     # Transformation into a NumPy array
166     weights = np.array(weights)
167
168     # -----

```

```

169 # Portfolio return
170 # -----
171 # mu_p = w^T mu
172 #
173 p_ret = np.dot(weights, mean_returns)
174
175 # -----
176 # Volatility
177 # -----
178 # sigma_p = sqrt(w^T Sigma w)
179 #
180 p_vol = np.sqrt(
181     np.dot(
182         weights.T,
183         np.dot(cov_matrix, weights)
184     )
185 )
186
187 # -----
188 # Sharpe Ratio
189 # -----
190 # R_f = 0
191 #
192 # Sharpe = mu_p / sigma_p
193 #
194 p_sharpe = p_ret / p_vol
195
196 # We return:
197 # [return, volatility, Sharpe]
198 return np.array([p_ret, p_vol, p_sharpe])
199
200 # -----
201 # Optimization objective function
202 # -----
203
204 def min_func_sharpe(weights):
205
206     # We want to MAXIMIZE the Sharpe Ratio.
207     # But scipy.optimize.minimize() seeks to MINIMIZE
208     # a function.
209     # Therefore:
210     # maximize(Sharpe)
211     # is equivalent to:
212     # minimize(-Sharpe)
213     #
214     return -portfolio_stats(weights)[2]
215
216 # =====
217 # OPTIMIZATION CONSTRAINTS
218 # =====
219
220 # -----
221 # Constraint: the sum of weights must equal 1
222 # -----
223 # scipy requires a constraint of the form:
224 # f(x) = 0
225 # We therefore define:
226 # sum(w_i) - 1 = 0
227 # which is equivalent to:
228 # sum(w_i) = 1
229 #

```

```

230 constraints = (
231     {
232         'type': 'eq',
233         'fun': lambda x: np.sum(x) - 1
234     }
235 )
236
237 # -----
238 # Weight constraints
239 # -----
240 # Each weight must satisfy:
241 # 0 <= w_i <= 1
242 # This means:
243 # - no short selling
244 # - no asset can have a weight greater than 100%
245 bounds = tuple(
246     (0, 1) for asset in range(len(assets))
247 )
248
249 # -----
250 # Algorithm starting point
251 # -----
252 # We start with an equally weighted portfolio:
253 #
254 # w = (25%, 25%, 25%, 25%)
255 #
256 initial_guess = len(assets) * [1. / len(assets)]
257
258 # =====
259 # SOLVING THE OPTIMIZATION PROBLEM
260 # =====
261
262 # We seek the portfolio that minimizes:
263 #
264 #           - mu_p
265 # min = -----
266 #           sigma_p
267 #
268 # which is equivalent to maximizing the Sharpe Ratio:
269 #
270 #           mu_p
271 # max= -----
272 #           sigma_p
273 #
274 # subject to:
275 # sum(w_i) = 1
276 # and:
277 # 0 <= w_i <= 1
278 #
279 opt_results = sco.minimize(
280     min_func_sharpe,
281     initial_guess,
282     method='SLSQP',
283     bounds=bounds,
284     constraints=constraints
285 )
286
287 # Retrieval of the optimal weights
288 optimal_weights = opt_results.x
289
290 # =====

```

```

291 # 5. PORTFOLIO VISUALIZATION
292 # =====
293
294 plt.figure(figsize=(10, 6))
295
296 # -----
297 # Scatter plot of the 10,000 simulated portfolios
298 # -----
299 # X-axis:
300 # portfolio volatility = risk
301 # Y-axis:
302 # expected portfolio return
303 # Color:
304 # Sharpe Ratio
305 #
306 plt.scatter(
307     results[0, :],      # Volatility
308     results[1, :],      # Return
309     c=results[2, :],    # Sharpe Ratio
310     cmap='viridis',
311     marker='o',
312     s=10,
313     alpha=0.3
314 )
315
316 # Color bar indicating the Sharpe Ratio value
317 plt.colorbar(
318     label='Annualized Sharpe Ratio'
319 )
320
321 # -----
322 # Addition of the optimal portfolio
323 # -----
324
325 # portfolio_stats(optimal_weights) returns:
326 #
327 # [return, volatility, Sharpe]
328 # Therefore:
329 # [1] = volatility
330 # [0] = return
331 #
332 plt.scatter(
333     portfolio_stats(optimal_weights)[1],
334     portfolio_stats(optimal_weights)[0],
335     color='red',
336     marker='*',
337     s=200,
338     label='Max Sharpe (Optimum)'
339 )
340
341 # Titles and axes of the graph
342 plt.title(
343     "Markowitz Portfolio \& Optimal Portfolio",
344     fontsize=12
345 )
346
347 plt.xlabel(
348     "Annualized Volatility (Risk)"
349 )
350
351 plt.ylabel(

```

```

352     "Annualized Expected Return"
353 )
354
355 # Legend
356 plt.legend()
357
358 # Grid
359 plt.grid(
360     True,
361     linestyle='--',
362     alpha=0.5
363 )
364
365 # Saving the graph
366 plt.savefig(
367     'frontiere_efficiente.png'
368 )
369
370 # Closing the figure
371 plt.close()
372
373 # =====
374 # 6. DISPLAYING THE RESULTS
375 # =====
376
377 print("-" * 40)
378 print("OPTIMAL ALLOCATION (MAX SHARPE):")
379 print("-" * 40)
380
381 # We associate each asset with its optimal weight.
382 for asset, weight in zip(assets, optimal_weights):
383
384     # :.2\% displays the weight as a
385     # percentage with two decimal places.
386     print(
387         f"Optimal weight for {asset} : {weight:.2\%}"
388     )
389
390 print("-" * 40)

```

3.1 Machine-Based Practical Work (Exercises)

Exercise 1 : The Minimum Variance Portfolio (Global Minimum Variance - GMV)

1. Write a Python function `min_func_variance(weights)` that returns only the calculated portfolio volatility.
2. Use `scipy.optimize.minimize` with the SLSQP algorithm, the same bounds and constraints, to extract the weights of the GMV portfolio.
3. Add this point to the previous graph (use a blue square marker, for example) to distinguish it from the Max Sharpe portfolio.

3.1.1 Solution

1. Place after `min_func_sharpe()` :

```

1 # -----
2 # Objective function: Global Minimum Variance Portfolio (GMV)
3 # -----
4
5 def min_func_variance(weights):
6
7     # We seek to MINIMIZE the portfolio volatility.
8     #
9     # portfolio_stats(weights) returns:
10    # [return, volatility, Sharpe]
11    #
12    # We therefore retrieve only the volatility:
13    return portfolio_stats(weights)[1]
14

```

2. Just after the optimization of the Max Sharpe portfolio :

```

1 # Retrieval of the optimal weights
2 optimal_weights = opt_results.x
3
4
5 # =====
6 # OPTIMIZATION OF THE GMV PORTFOLIO
7 # =====
8
9 # We now seek to directly minimize
10 # the portfolio volatility.
11 #
12 # Constraints:
13 #
14 # sum(w_i) = 1
15 #
16 # and:
17 #
18 # 0 <= w_i <= 1
19
20 gmv_results = sco.minimize(
21     min_func_variance,
22     initial_guess,
23     method='SLSQP',
24     bounds=bounds,
25     constraints=constraints
26 )

```

```

27
28
29 # Retrieval of the optimal weights of the GMV portfolio
30 gmv_weights = gmv_results.x

```

`gmv_weights` contains the weights of the Global Minimum Variance Portfolio.

3. After the Max Sharpe portfolio, add the following code :

```

1 # -----
2 # Addition of the GMV portfolio
3 # -----
4
5 # portfolio_stats(gmv_weights) returns:
6 #
7 # [return, volatility, Sharpe]
8 #
9 # Therefore:
10 #
11 # [1] = volatility
12 # [0] = return
13
14 plt.scatter(
15     portfolio_stats(gmv_weights)[1],
16     portfolio_stats(gmv_weights)[0],
17     color='blue',
18     marker='s',
19     s=100,
20     label='GMV (Minimum Variance)'
21 )

```

Exercise 2 : Integration of Allocation Limits (Regulatory Weight Constraints)

1. Modify the `bounds` argument of the optimizer so that no individual asset can represent more than 40% of the total portfolio, and none can fall below the minimum threshold of 5%.
2. Solve the Max Sharpe problem again under this geometric constraint.
3. Verify with a `print` that all calculated weights strictly satisfy :

$$0.05 \leq w_i \leq 0.40 \quad \forall i$$

Solution

1. Replace,

```

1 bounds = tuple(
2     (0, 1) for asset in range(len(assets))
3 )

```

with

```

1 bounds = tuple(
2     (0.05, 0.40) for asset in range(len(assets))
3 )

```

2. Run the Max Sharpe optimization again

```

1 # =====
2 # SOLVING THE MAX SHARPE PROBLEM UNDER CONSTRAINTS
3 # =====

```

```

4
5 opt_results = sco.minimize(
6     min_func_sharpe,
7     initial_guess,
8     method='SLSQP',
9     bounds=bounds,
10    constraints=constraints
11 )
12
13 optimal_weights = opt_results.x

```

3. Add the following code :

```

1 # =====
2 # CONSTRAINT VERIFICATION
3 # =====
4
5 print("-" * 40)
6 print("CONSTRAINT VERIFICATION:")
7 print("-" * 40)
8
9 for asset, weight in zip(assets, optimal_weights):
10
11     print(
12         f"{asset} : {weight:.2\\%} "
13         f"| Satisfies bounds : {0.05 <= weight <= 0.40}"
14     )
15
16 print("-" * 40)
17
18 print(
19     "All weights satisfy the constraints:",
20     np.all((optimal_weights >= 0.05) &
21            (optimal_weights <= 0.40))
22 )

```

Exercise 3 – How Stable Is a Markowitz Portfolio ?

Consider the following six assets :

AAPL, MSFT, AMZN, NVDA, JPM, XOM.

Use daily adjusted closing prices over the last five years and Assume :

$$w_i \geq 0, \quad \sum_{i=1}^6 w_i = 1, \quad R_f = 0.$$

A – Maximum Sharpe Ratio Portfolio

1. Compute daily logarithmic returns :

$$r_{i,t} = \ln \left(\frac{P_{i,t}}{P_{i,t-1}} \right).$$

2. Annualize the expected returns and covariance matrix :

$$\mu_{\text{annuel}} = 252 \mu_{\text{daily}},$$

$$\Sigma_{\text{annual}} = 252 \Sigma_{\text{daily}}.$$

3. Using the SLSQP method, determine the portfolio maximizing :

$$\max_w \frac{w^\top \mu}{\sqrt{w^\top \Sigma w}}$$

subject to :

$$\sum_{i=1}^6 w_i = 1, \quad w_i \geq 0.$$

4. Report :
- the optimal weights ;
 - the expected return ;
 - the volatility ;
 - the Sharpe ratio.

B – Does the Estimation Period Matter ?

1. Repeat the optimization using three different estimation windows :

$$\boxed{12 \text{ months}} \quad \boxed{36 \text{ months}} \quad \boxed{60 \text{ months}}$$

For each window, compute the maximum-Sharpe portfolio.

2. Present the weights in the following table :

	<i>AAPL</i>	<i>MSFT</i>	<i>AMZN</i>	<i>NVDA</i>	<i>JPM</i>	<i>XOM</i>
12 months						
36 months						
60 months						

Then create a graph comparing the three allocations.

C – Interpretation

Answer the following questions.

1. Question 1.
 - Are the three optimal portfolios similar or different ?
 - Which assets have the most stable weights ?
 - Which assets have the largest changes ?
2. Question 2.
 - Why can changing the estimation period lead to different optimal portfolios ?
 - Discuss the role of :

$$\mu \quad \text{and} \quad \Sigma.$$

3. Question 3.

Suppose that two estimation periods produce very different portfolios. Does this necessarily mean that the Markowitz model is wrong ? Explain your answer.

4. Question 4.

In practice, why might an investor prefer to impose additional allocation constraints such as

$$0.05 \leq w_i \leq 0.30$$

instead of using the unconstrained optimal portfolio?